Third Collection of Python Problems

Ilkka Kokkarinen

Chang School of Continuing Education
Toronto Metropolitan University

Version of October 20, 2026

This document contains the specifications for a third collection of Python problems, created by Ilkka Kokkarinen starting in March 2023 to augment the original set of 109 Python Problems for the course CCPS 109 Computer Science I at Chang School of Continuing Education, Toronto Metropolitan University (formerly Ryerson University).

Same as in the original problem collection, these problems are listed roughly in order of increasing difficulty. The reader shouldn't get too hung up on that, since that estimate is basically a gut feeling anyway and depends heavily on the reader's propensities and interests. The solutions to these additional problems range from straightforward loops to convoluted branching recursive backtracking searches that require clever optimizations to prune the search tree enough to keep the test suite's total running time within the twenty-second limit.

The rules for solving and testing these problems are the same as they were for the original 109 problems. You must implement all your functions in a single source code file that must be named `labs109.py`, the same way as you wrote your solutions to the original 109 problems. The automated test suite to check that your solutions are correct, along with all associated files, is available in the GitHub repository `ikokkari/PythonProblems`.

Python coders of all levels will hopefully find something interesting in this collection. The author believes there is room on Earth for all races to live, prosper and get strong in coding while having a good time solving these problems. Same as with the original 109 Python problems, if a problem doesn't spark joy for you, please don't get stuck with that problem, but skip it and move on to the next problem that interests you. Life is too short to waste on stupid problems.

Table of Contents

1. Multiply and sort **7**
2. Approval voting **8**
3. Vigenère cipher **9**
4. Once you go down, you can't come up again **10**
5. Baum–Sweet sequence **11**
6. Damm those errors **12**
7. Tanton necklace **13**
8. Bug in a line **14**
9. Friendship paradox **15**
10. Right turn at Albuquerque **16**
11. Generalized Fibonacci sequence **17**
12. Pollard's rho integer factorization **18**
13. Bays–Durham shuffle **19**
14. Double-ended pop **20**
15. First singleton **21**
16. Candy sharing game **22**
17. Haar wavelet transform: Encode **23**
18. Haar wavelet transform: Decode **24**
19. Shell sort **25**
20. Mixed type sorting **26**
21. The nine lives of Meaowsephus **27**
22. Maximum repeated suffix **28**
23. Count slices with given sum **29**
24. First sublist whose sum is a multiple of k **30**
25. Ones mule, zeros pool **31**
26. The backwards god **32**
27. Klee's measure problem **33**
28. Poker test of randomness **34**
29. Three-way partition **35**
30. Primality buildup **36**
31. Goldbach verification **37**
32. A square and twice a prime **38**
33. Pseudo-Smarandache function **39**

34. Conway’s subprime sequence **40**
35. Rowland's prime pump **41**
36. Lehmer code: encoding **42**
37. Sum of square roots **43**
38. Night shift **44**
39. Infinite self-square integer **45**
40. Strahler numbers **46**
41. Loopless walk **47**
42. Maximum number of blocks during shutdown **48**
43. List of all factors **49**
44. Fourth seat opening **50**
45. Convert a fraction to a simple continued fraction **51**
46. Convert a simple continued fraction to a fraction **52**
47. Sturm und drangle **53**
48. Rational roots of a polynomial **54**
49. Multiple winner election **55**
50. Spiral walk **56**
51. Equal sum partition **57**
52. Maximum interval clique **58**
53. Maximum overlap intervals **59**
54. Split at None **60**
55. Chunk sorting **61**
56. Merge the two biggest **62**
57. Lehmer code: decoding **63**
58. Bayesian updating of dice **64**
59. Haircut **65**
60. Kadane in triplicate **66**
61. Yellowstone sequence **67**
62. Largest rectangle in a histogram **68**
63. Frobenius nuggets **69**
64. Postage stamp problem **70**
65. Knaves of the round table **71**
66. Six of this, half a dozen of the other **72**
67. Factoradical dudes **73**
68. Tarjan's strongly connected components **74**
69. Copeland harder **75**
70. Instant-runoff voting **76**
71. Cousin explainer **77**
72. One-dimensional random walk **78**
73. Knight watch **79**
74. Longest Fibonacci-like subsequence **80**

75. Maximize exam confusion **81**
76. Burrows–Wheeler transform **82**
77. Bron–Kerbosch maximal clique enumeration **83**
78. Queue jockeys **84**
79. Distance from land **85**
80. Smallie Bigs **86**
81. Gauss circle **87**
82. Zeckendorf decoding **88**
83. Sultan's daughter **89**
84. Limited swap sorting **90**
85. Average run length test of randomness **91**
86. Count increasing paths **92**
87. Cocke–Younger–Kasami parsing **93**
88. Avoidant subset of points on a circular line **94**
89. Ehrenfeucht–Mycielski sequence **95**
90. Diamond sequence **96**
91. Minimize largest triangle **97**
92. Cutting sticks **98**
93. Optimal text filling **99**
94. S-expression evaluator **100**
95. Expand recursively run-length encoded string **101**
96. Longest common strictly increasing subsequence **102**
97. Maximum palindromic integer **103**
98. Optimal palindromic split **104**
99. Semiconnected guys **105**
100. Falling squares **106**
101. Tchuka Ruma **107**
102. Slater–Vélez sequence **108**
103. Odds and evens **109**
104. The prodigal sequence **110**
105. Descending suffix game **111**
106. The Borgesian dictionary **112**
107. The Dictorian borgesitory **113**
108. The art of strategic sacrifice **114**
109. Colonel Blotto and the Spymaster **115**
Bonus problem 110: The way of a man with array **116**
Bonus problem 111: Levine’s sequence **117**
Bonus problem 112: Post correspondence problem **118**
Bonus problem 113: Optimal bridge placement **119**
Bonus problem 114: No dogs, no masters **120**
Bonus problem 115: Car match game **121**

Bonus problem 116: Liquid sort **122**
Bonus problem 117: Count bowling paths **123**
Bonus problem 118: Expiring cookies **124**
Bonus problem 119: Sonata pattern matching **125**
Bonus problem 120: Fox and hounds **126**
Bonus problem 121: Toads and frogs **127**
Bonus problem 122: Unamerican gladiators **128**
Bonus problem 123: Life is like a box of dominoes **129**

1. Multiply and sort

`def multiply_and_sort(n, multiplier):`

The following problem, unearthed from the post “Double (or triple) and sort” in Michael Lugo's blog “God Plays Dice” (“where randomness meets divine intervention”), explores an iterated numerical transformation with curious properties and general behaviour. Starting with a positive integer `n`, multiply the current number by the given multiplier, then sort the resulting digits in ascending order to get the next number in the sequence. Leading zeros, being the shy digits that they are, quietly disappear from the front.

For example, with *n* = 513 and `multiplier` = 2, the product 1026 becomes 0126 after sorting, which simplifies to 126 (the zero, true to form, showing itself out). This process continues until an integer repeats in the sequence, at which point the sequence stops. Your task in writing this function is to return the largest integer encountered during this mathematical game of multiply-and-tidy-up.

n	multiplier	Expected result
2	5	4456
3	4	2667
5	5	4456
20	7	15556688
31	7	12345678888888889

As one commenter on the original post pointed out, this process seems to always terminate for multipliers up to 20, but then freezes for some multipliers from 21 onwards and behaves well for the others. Interested readers might want to explore this behaviour further and prove necessary and sufficient conditions for termination. The automated tester will, of course, give your function only numbers for which this sequence will terminate in a reasonable time.

2. Approval voting

```
def approval_voting(ballots):
```

Approval voting is an alternative to the traditional first-past-the-post single winner election scheme. Instead of voting for one favoured candidate, every voter marks every candidate on the ballot as either approved or disapproved. The candidate with the most approvals across all `ballots` wins.

For this problem, assume that the *n* candidates have been numbered 0, …, *n* – 1, and each ballot is a text string of length *n* whose each character is either `'Y'` or `'N'` depending on whether that voter goes “yea” or “nay” for that candidate. This function should tally up the approvals for each candidate and return the index of the winning candidate. If multiple candidates receive the same number of approvals, your function should resolve the tie for the candidate with the lowest position index.

`ballots`	Expected result
`['NYN', 'NYN', 'NYN', 'YNN', 'YYN']`	`1`
`['NYY', 'NYY', 'YYN', 'NYY', 'NNY', 'NYY']`	`1`
`['NYNYN', 'NYNYY', 'YYYNY', 'YYYYN', 'YNNNY', 'YNNNY', 'NYYNN', 'YNNYN']`	`0`

3. Vigenère cipher

```
def vigenere(text, key, direction):
```

The Vigenère cipher is a cryptographic method dating back almost 500 years. Simple enough to allow both encryption and decryption to be performed by hand even under battlefield conditions, this method was nearly impossible to crack with the methods of its era by someone who did not have the agreed encryption `key`, at least within the time limit when the encrypted information would have made a difference on the battlefield.

Using the modern English alphabet, the Vigenère method encrypts the given `text` one character at a time, replacing each character with the character shifted right in the alphabet (wrapping around after z) by the number of positions given by the corresponding key character. The character `a` denotes a shift of 0, `b` denotes a shift of 1, and so on, up to `z`, which denotes a shift of 25. If the `key` is shorter than the text, the `key` is concatenated with itself as many times as needed to stretch it to sufficient length. To decrypt the `text` with the same `key`, simply shift each character that same amount in the opposite direction.

Given the `text` and `key`, in this problem both guaranteed to consist of the 26 lowercase letters of English only, this function should perform both encryption (if `direction` equals +1) and decryption (if `direction` equals –1) of the `text` with the given `key`.

text	key	direction	Expected result
'locoedturse'	'adsum'	+1	'lruiqdwmlee'
'locoedturse'	'adsum'	-1	'llkusdqcxge'
'marorcotoin'	'kafiz'	+1	'wawwqmoywhx'
'marorcotoin'	'kafiz'	-1	'camgssoogjd'

It is also easy to see that the Vigenère cipher includes both the Caesar cipher and rot-13 as trivial special cases, with a single-letter key in both cases.

4. Once you go down, you can't come up again

```
def is_organ_pipe(items):
```

We define a list of `items` to have an **organ-pipe shape** if its elements are first **strictly ascending** to some pinnacle value, after which the elements are **strictly descending**. The two halves of the organ pipe don't need to be the same length, so `[3, 5, 8, 9, 10, 4, 2, 1]` qualifies as having an organ-pipe shape. However, we do require both sides of the organ pipe to consist of at least one step, which means that a sorted list that is entirely strictly ascending or strictly descending does not qualify as an organ pipe in this problem. This function should determine whether the given list of `items` has an organ-pipe shape.

items	Expected result
`[5, 3]`	`False`
`[1, 2, 1]`	`True`
`[1, 2, 2, 1]`	`False`
`[1, 3, 4, 6, 3, 0]`	`True`
`[0, 2, 1, 4, 6, 4, 1]`	`False`
`[0, 2, 3, 4, 5, 8, 9, 12, 9, 4]`	`True`

The title of this problem contains the key insight to make this test simple and efficient.

5. Baum–Sweet sequence

```
def baum_sweet(n):
```

The Baum–Sweet sequence is defined with a simple function that, given a nonnegative integer parameter *n*, returns 1 if the binary representation of *n* does not contain any odd-length sequences of zero bits, and returns 0 if the binary representation contains at least one such sequence of zeros. Note that 1 is an odd number. This function should compute and return the value in the Baum–Sweet sequence for the given integer parameter *n*.

You can iterate through the bits of n with a loop that repeatedly computes the remainder of *n* modulo 2 to extract the lowest bit, and then divides *n* by two to chop off that bit. Alternatively, you can use the Python built-in function `bin` that returns the binary string representation of its integer parameter *n*, prefixed with `'0b'` to indicate base-2 encoding.

n	Expected result
0	0
1	1
7	1
67	1
69	0

6. Damm those errors

`def damm_checksum(digits):`

Check digits are used for credit card numbers and other digit-string identifiers to catch errors humans tend to make when entering boring, meaningless digit sequences. When an identifier digit string is issued, the check digit algorithm computes a single-digit checksum that becomes part of the identifier. The checksum digit lets you check the validity of the preceding digits on the spot, before proceeding to the identifier's authenticity checks.

In this problem, we shall implement the Damm checksum algorithm. We can implement this algorithm as a simple **state machine** that processes each digit exactly once. Still, thanks to its clever design, it can detect any single-digit change or the transposition of any two consecutive digits. Damm's algorithm maintains an **internal state**, an integer between 0 and 9 inclusive, initially set to 0. The algorithm then processes the digits of the given digit string one at a time from left to right.

The algorithm uses a two-dimensional state-machine transition table that can be constructed in many ways. In this problem, we will use the one Damm originally computed, as given on the Wikipedia page describing the algorithm. If the algorithm's current internal state is `s`, the state after processing the next digit `d` will be `table[s][d]`, read from the transition table using the current state as the row index and the current digit as the column index. Your function should perform this internal state update for each digit, and return the resulting internal state of the algorithm as the answer.

digits	Expected result
'572'	4
'251'	6
'00759'	9
'2770753'	0

7. Tanton necklace

```
def tanton_necklace(beads, m):
```

The Australian math educator James Tanton has the seemingly endless gift of creating deep problems simple enough to state in a single tweet, in a few almost throwaway sentences, yet clear enough for any schoolboy to understand on the spot.

Inspired by this recent tweet, consider a cyclic **necklace** of 2*n* beads, each either purple or orange, with exactly *n* beads of each colour. This function should count how many contiguous **balanced sublists** of 2*m* beads consist of exactly *m* purple beads and *m* orange beads, and return that count. (In the table below, the starting positions of these sublists are marked with a green background.) Every necklace given to this function is nonempty and valid according to this specification.

To make this function efficient for thousands of pseudorandom test cases, you should devise a way to quickly determine whether the sublist of 2*m* starting from position *i* is balanced. Of course, you could iterate through all 2*n* possible starting positions, and for each such position, extract the 2*m* beads and count how many of each colour there are, incrementing the tally whenever the sublist contains exactly *m* of each colour of bead. However, this seems like a very "Shlemiel" way to solve this problem. Instead, note that if you have already counted the number of beads of each colour in the subsequence of length 2*m* starting at position *i*, you can update these counts for the subsequence starting at position *i* + 1 in constant time.

beads	m	Expected result
'ooopppp'	2	2
'ppopoooop'	3	4
'oooooppopopooppppppp'	3	6

As one commenter to Tanton's original tweet pointed out, at least one balanced subsequence with exactly *m* purple and exactly *m* orange beads is guaranteed to exist for any *m*, regardless of how the original purple and orange beads are distributed over the entire necklace. Discrete math enjoyers reading this can try their hand at proving this interesting Ramsey theory topic.

8. Bug in a line

```
def bug_in_a_line(board):
```

The following problem is from Peter Winkler's "Mathematical Puzzles: A Connoisseur's Collection," where this simple system is shown to have interesting mathematical properties over all possible boards. Here we shall content ourselves with running the system on the given starting `board`. A smol cartoon bug stands at position 0 of the `board`, a string whose characters are traffic lights: `R`ed, `Y`ellow, or `G`reen. The colour of the traffic light in the cell that the bug is at determines its next move:

- If the current light is **green**, the bug flips that light to be **yellow**, and moves one step **right**.
- If the current light is **yellow**, the bug flips that light to be **red**, and moves one step **right**.
- If the current light is **red**, the bug flips that light to be **green**, and moves one step **left**.

The bug therefore moves left roughly one-third of the time and right roughly two-thirds of the time. This function should iterate through the bug's steps on the board until it falls off either edge, and return the total number of steps it took to fall off. You will first convert the `board` to a list instead of an immutable string so you can locally modify the traffic light the bug flips in constant time, regardless of `board` length.

`board`	Expected result
`'GGR'`	`5`
`'RRGY'`	`1`
`'GYRGYYRRRR'`	`26`
`'GGYRRYYYYRYRYG'`	`34`

The bug cannot wander forever on a finite `board`; it must eventually fall off one end or the other. If the bug traverses a finite board infinitely long, it must visit some cells infinitely often. Let k be the lowest position index of these infinitely often visited cells. Since that cell is red during one-third of these visits, the bug would also visit cell $k - 1$ infinitely often, contradicting the assumption that k was the lowest position index with that property.

9. Friendship paradox

```
def friendship_paradox(friends):
```

Each person in a group of n people numbered 0, ..., $n - 1$ has at least one friend in the group, with `friends[i]` giving the list of friends of person i. Friendship or its absence is always symmetric between any two people. (After all, unrequited friendship belongs in middle school diary entries of overly dramatic emo theatre kids, not in a serious and manly study of graph theory.)

This problem investigates a curious phenomenon known in the literature as the friendship paradox: on average, an individual's friends have more friends than that individual. While this may sound like a manifestation of social anxiety and everyone else seemingly having cooler lives than you, this phenomenon has a rigorous mathematical basis in that more popular people appear as someone's friends more frequently than less popular people, which skews the average number of friends that your friends have compared to your own friendship count.

Given the list of lists of `friends` for the people in the group, this function should compute and return two numerical quantities as a two-element tuple: the average number of friends per person, and the average number of friends among the friends per the average person. Both quantities should be computed and returned as exact `Fraction` objects, because real mathematicians don't round their social calculations.

`friends`	Expected result
`[[1], [0, 2], [1]]`	`(Fraction(4, 3), Fraction(5, 3))`
`[[1, 3], [2, 0, 3], [1, 3], [1, 0, 2]]`	`(Fraction(5, 2), Fraction(8, 3))`
`[[1, 4, 3, 2], [3, 2, 4, 0], [1, 0, 3], [4, 1, 0, 2], [0, 3, 1]]`	`(Fraction(18, 5), Fraction(37, 10))`

As a coding challenge, the enjoyers of Python **generator expressions** can try to write the part of the function that computes the sum of friends that are friends of a particular person as a one-liner.

10. Right turn at Albuquerque

```
def albuquerque_stretch(text):
```

This cute little problem, suitable for a coding exercise for a Python intro course, is “Albuquerque challenge,” recently posted on Code Golf Stack Exchange. Still, of course, we'll aim for correctness and clarity rather than the source code's extreme brevity. This function should first set the `result` to an empty string, then loop through the characters `c` of the `text` from left to right. At the first occurrence of that character `c`, simply append that character to the `result`. From the second occurrence onwards, append the substring of the original `text` spanning from the most recent occurrence of `c` to its current occurrence, both endpoints inclusive, to the `result`.

You can trace the steps of the first test case `'success'` either by yourself or by following its steps on the original web page linked above for a concrete explanation of these rules. It is probably easiest to use a Python dictionary to keep track of the characters you have already seen in the loop, each character associated with the position of its previous occurrence in the `text`.

`text`	Expected result
`'success'`	`'succcesuccesss'`
`'albuquerque'`	`'albuququerquerquerquerque'`
`'glarrota'`	`'glarrrotarrota'`
`'dontgangseedmmf'`	`'dontgantgangangseeedontgangseedmmmf'`

11. Generalized Fibonacci sequence

```
def generalized_fibonacci(multipliers, n):
```

The Fibonacci sequence is a dusty cliché in teaching computer science, usually presented as a silly exponential-time recursion, but better implemented as a loop that produces elements one at a time based on previous elements. Since we are going to write this function, we might as well implement a general version that can produce not just the terms of the plain old Fibonacci sequence, but any generalized Lucas sequence where each term F_i is computed as a sum of some m immediately preceding values F_{i-1}, F_{i-2}, ..., F_{i-m}, each preceding term multiplied by the given fixed multiplier a_j applied to the term F_{i-j}. For example, the classic Fibonacci sequence is defined by the recursive equation $F_i = F_{i-1} + F_{i-2}$, where the multipliers of the two preceding terms are $a_1 = a_2 = 1$. Pell numbers are defined with the equation $F_i = 2F_{i-1} + F_{i-2}$, with initial terms $a_1 = 2$ and $a_2 = 1$.

Given the `multipliers` as a tuple of m elements, compute the term in position `n` of the sequence. Position numbering starts from 0, and the first terms in positions 0, ..., $m - 1$ of the sequence all equal 1. The order of `multipliers` is such that the last multiplier in position `-1` is applied to the term immediately preceding the current term being computed, the second-to-last multiplier in position `-2` is applied to the term before that term, and so on.

`multipliers`	`n`	Expected result
`(1, 1)`	`5`	`8`
`(1, 2)`	`8`	`577`
`(4, 4, -3, -7, 5, 0, 7)`	`31`	`2579538296491128384859`
`(7, -7, 9, 9, -7, 1, -3, 9, 4)`	`35`	`416464779917747725480`

12. Pollard's rho integer factorization

```
def pollard_rho(n):
```

Pollard's rho algorithm is an ingenious algorithm for quickly factoring large integers. Based on efficient cycle detection with the idea of **Floyd's tortoise and hare algorithm**, the tactically chosen function $g(x) = (x^2 + 1) \bmod n$ is iterated starting from $x = 2$ until a cycle is found; the final value of this cycle then magically reveals one of the prime factors of n.

In this problem, you should implement the algorithm from the pseudocode given on the linked Wikipedia page. Your function should return the list of prime factors of n in ascending sorted order. To keep this simple, the automated tester is guaranteed to give your function only values of n that are known **semiprimes**, that is, satisfy $n = pq$ for some two prime numbers p and q. Therefore, once your function has found one of these two prime factors of n, the other prime factor is given by a simple integer division.

n	Expected result
10	[2, 5]
79413038221179407	[26797907, 2963404501]
84997913392275761	[25537943, 3328299127]
65901324277161883	[22238383, 2963404501]
78644668293912529	[22682923, 3467131123]

13. Bays–Durham shuffle

```
def bays_durham_shuffle(item_stream, k):
```

The quality and period of the sequence of pseudorandom numbers produced by any simple random number generator can be massively improved with the ingenious Bays–Durham shuffle algorithm, which is simple enough to implement here in a handful of lines of Python code.

This function receives a Python **iterator** named `item_stream` that produces a sequence of nonnegative integers, and it should apply the Bays–Durham shuffle to this sequence as follows. Unlike in the other problems that you have seen in this and previous two Python problem collections, this iterator does not support **random access indexing** at arbitrary positions. Still, it produces elements one at a time when you call `next()` on it, raising `StopIteration` when it has no elements left.

Use the first `k` items from the stream to build a `table` of *k* elements, and initialize the `result` list to be empty. Then loop through the remaining *n* – *k* items from the stream, and perform the following operation for each such item *e*: replace the element *x* currently in the position (*e* mod *k)* with the item *e*, and append the removed element *x* to the result list. For example, suppose that *k* = 4 and the current `table` contains the values `[17, 42, 99, 5]`, and the next item from the stream equals 55. Compute the table position with the formula 55 % 4 = 3, replace the element 5 in that position with 55, and append that replaced element 5 to the result list, giving you the new `table` with values `[17, 42, 99, 55]`. This function should return the list of *n* – *k* values computed from the stream in this manner, after the `table` of the first *k* elements of the stream.

(Inquiring minds would also want to know, of course, what would happen if the Bays–Durham shuffle were iteratively applied to the random sequence already shuffled in that manner.)

14. Double-ended pop

```
def double_ended_pop(items, k):
```

You are given a list of `items`, each one a nonnegative integer, and are allowed to do the following operation `k` times: remove an element from either the beginning or the end of the `items` list. This function should return the largest possible sum of items that can be built up with *k* such removal operations.

items	k	Expected result
[13, 0, 9, 11, 6, 8, 18, 13, 5, 11]	4	47
[7, 11, 17, 10, 5, 12, 10, 0, 1, 1, 14, 2, 12]	4	47
[17, 9, 0, 15, 19, 18, 7, 10, 20, 2, 0, 0, 20, 13, 3, 9, 14, 2, 2]	7	85
[21, 14, 24, 0, 21, 7, 21, 1, 4, 11, 18, 20, 0, 6, 0, 9, 21, 12, 16, 20]	5	95

15. First singleton

```
def first_singleton(text):
```

This function should find and return the first character in the `text` that occurs in that `text` exactly once. If the `text` does not contain any singleton characters, return `None`.

Of course, your function has to examine every character of the `text` before it can return the answer with absolute confidence. However, with the aid of Python sets and dictionaries to keep track of the characters that you have seen once or more than once, your function should be a single for-loop through the `text` that processes each character in constant time, and thus be blazingly fast even when tested with thousands of strings that contain thousands of characters each.

`text`	Expected result
`'gjexx'`	`'g'`
`'bababa'`	`None`
`'dddhmsddhdsmdrdhdddnddrs'`	`'n'`
`'fzmfomfpzmzfpomxfculzmfcqm'`	`'x'`

16. Candy sharing game

```
def candy_split(candies):
```

This problem from the 2003 Greater New York Region ACM Collegiate coding contest is similar to the candy-sharing problem in our original 109 Python Problems collection, with a different rule for distributing the candies. Several students sit in a circle, and each student initially has an even number of `candies`. On the teacher's mark, all students simultaneously give exactly half of their candies to the student sitting immediately to their right. Between rounds, the teacher gives each student with an odd number of candies one extra piece to ensure that every student always has an even number.

This system differs from the candy-sharing problem in the original 109 Python problems collection because the total number of candies does not remain constant; the teacher keeps adding new candies to maintain even parity. However, this system can still be proven to reach an equilibrium state in which each student has the same even number of candies. This function should return a two-tuple: the number of rounds needed to reach equilibrium, followed by the number of candies each student ends up with.

`candies`	Expected result
`[2, 4, 0]`	`(3, 4)`
`[4, 4, 2]`	`(1, 4)`
`[8, 8, 2, 2, 2]`	`(7, 8)`
`[18, 6, 6, 8, 6, 14, 10, 22, 22]`	`(16, 22)`
`[50, 22, 56, 2, 24, 30, 14, 6, 24, 46, 40, 46, 36, 46, 24, 12, 14, 30, 32, 10, 16, 44]`	`(40, 44)`

17. Haar wavelet transform: Encode

```
def haar_wavelet_encode(signal):
```

In **signal processing**, we transform a series of observed signals into a different space so we can compute certain operations more efficiently. We shall leave the theoretical underpinnings of these transformations for an advanced mathematics course. Instead, as an exercise in Python list processing with simple recursion, we will implement the discrete Haar wavelet transform, since it avoids complex or floating-point numbers and can be implemented with straightforward integer arithmetic.

For simplicity, we assume that the `signal` is a list of integers whose length is some power of two. To perform the transform, split the `signal` into pairs of consecutive elements and create a list of pairwise sums and differences. (The classical Haar transform also divides all sums and differences by two, but we shall skip that step to keep all values here as integers.)

For example, if the `signal` equals `[2, 3, -1, 4, 7, 2, 4, 4]`, the resulting list of pairwise sums is `[5, 3, 9, 8]` and the list of pairwise differences is `[-1, -5, 5, 0]`. Compute the Haar wavelet transform of the list of pairwise sums recursively, and concatenate the transformed pairwise sums with the list of pairwise differences as it is without transform. The base case of the recursion is the singleton `signal` list that just transforms to itself.

`signal`	Expected result
`[1, -5, -1, 0]`	`[-5, -3, 6, -1]`
`[4, -1, 2, 1]`	`[6, 0, 5, 1]`
`[-6, 4, -3, -8, 3, -2, 8, -5]`	`[-9, -17, 9, -2, -10, 5, 5, 13]`
`[2, 7, -3, 10, 4, 6, -7, 6, -10, 6, 7, 1, -3, 2, 5, -8]`	`[25, 25, 7, 8, 2, 11, -12, 2, -5, -13, -2, -13, -16, 6, -5, 13]`

The Haar wavelet transform does not lose any information about the original `signal`, as shown in the following problem, which asks you to restore the `signal` from its Haar wavelet coefficients. Before restoration, you can use these coefficients for compression (after smoothing and discarding small wavelet coefficients, the resulting coefficients compress better than the original noisy signal), feature extraction, edge detection, and signal analysis.

18. Haar wavelet transform: Decode

```
def haar_wavelet_decode(coefficients):
```

This function is the inverse of the discrete Haar wavelet transform encoding function of the previous problem. Given the list of `coefficients` produced by the original Haar wavelet transform, restore the original `signal` that gave these `coefficients`. Again, for simplicity, you may assume that the length of the `coefficients` list is some power of two.

`coefficients`	Expected result
`[1, -5, -1, 0]`	`[-2, 0, 1, 2]`
`[-6, -4, 6, -2]`	`[0, -5, -2, 1]`
`[5, 2, 3, 0, 5, -1, 0, 6]`	`[4, -1, -1, 1, 0, 1, 3, -2]`
`[3, 8, -7, -5, 11, -4, 1, -12, -8, -9, -7, 0, -5, -9, -6, 11]`	`[-2, 7, -8, 2, -3, 4, 2, 3, -4, 2, -6, 4, -6, 1, 9, -2]`

19. Shell sort

```
def shell_sort(permutation, gaps):
```

Most computer science students first study and implement insertion sort as a serious sorting algorithm. Insertion sort is provably optimal among all comparison sort algorithms that are constrained to compare and swap adjacent elements, as it will perform exactly as many comparisons and swaps as are necessary and sufficient to sort the list with such swaps.

Faster sorting algorithms must compare and swap elements across longer distances in the list to quickly move them closer to their final resting places. Shell sort generalizes insertion sort and operates in multiple rounds, each time using a different gap value from the `gaps` sequence in decreasing order. In each round, loop through positions from the current gap value g up to $n - 1$, and repeatedly compare and swap the element in that position with the element located g steps before in the list until either the gap g would lead past the beginning of the array, or the element g steps before the current position is less than or equal to the current element. Since the last gap size is always one, the last round performs an ordinary insertion sort that is guaranteed to sort the list regardless of how the previous rounds have rearranged the elements.

This function should implement the shell sort algorithm as described in the pseudocode on the linked Wikipedia page. However, instead of returning the sorted array, as **proof of work** that you genuinely did implement the actual Shell sort algorithm without any underhanded activity, your function should instead return the two-tuple `(comparisons, swaps)` of how many element comparisons and element swaps were performed during the algorithm for the given `permutation` of integers 0, ..., $n - 1$ using the given sequence of `gaps`.

`permutation`	`gaps`	Expected result
`[5, 4, 1, 2, 0, 3]`	`[1, 3]`	`(12, 7)`
`[6, 0, 5, 1, 3, 4, 2]`	`[1, 3, 5]`	`(17, 8)`
`[9, 7, 3, 5, 0, 1, 12, 6, 4, 2, 10, 8, 11]`	`[1, 2, 3, 4, 6, 8, 9, 12]`	`(70, 15)`
`[13, 7, 11, 9, 10, 4, 8, 14, 3, 5, 0, 2, 12, 6, 1]`	`[1, 4, 9]`	`(59, 35)`

20. Mixed type sorting

```
def mixed_sort(items):
```

Sorting a list of `items` implicitly requires that all elements are **pairwise comparable**, so they form a total order; for any two items *x* and *y*, we can always tell which is smaller whenever they are not equal. However, the general sorting problem can be generalized to **partial orders** by loosening the expectation for the resulting list. Whenever two original items *x* and *y* are not comparable, their relative placement in the sorted list is irrelevant.

For an interesting coding challenge comparing these two approaches, your task is to return a sorted list of original items (both integers and text strings) so that all integers end up in the same positions that originally contained an integer. All text strings should end up in the same positions that originally contained a text string. To determine the type of an element `x` in the list, use the Python built-in function `type` in something like `type(x) == int`.

items	Expected result
`[12, -14, 'cashoo', 'konia', 4, 'goldi', 4, 14, 'yockel']`	`[-14, 4, 'cashoo', 'goldi', 4, 'konia', 12, 14, 'yockel']`
`['gratae', -11, 12, -9, 4, -1, 'lupus', 13, 10]`	`['gratae', -11, -9, -1, 4, 10, 'lupus', 12, 13]`
`['imbute', 2, -2, 'kroon', 17, 0, 'hagdin', 'rerent', 'alidad', 'cantic']`	`['alidad', -2, 0, 'cantic', 2, 17, 'hagdin', 'imbute', 'kroon', 'rerent']`
`[-10, 15, -23, 16, -1, 'gasify', 'osiers', -22, -6, 1, -13, -20, 'fusel', 12, 20, 'axine', 17]`	`[-23, -22, -20, -13, -10, 'axine', 'fusel', -6, -1, 1, 12, 15, 'gasify', 16, 17, 'osiers', 20]`

21. The nine lives of Meaowsephus

```
def meaowsephus(lives, k):
```

The original *109 Python Problems* collection featured the classic Josephus problem, where a group of men around a circle get unalived one at a time according to a cruel discipline they all agreed to follow. However, we can generalize this problem into a more adorable form by replacing sweaty men with swords and sandals with cute kittens around the circle.

The kittens are numbered 0, …, n – 1. Each kitten initially has a certain number of `lives` remaining. The elimination process starts at the beginning of the circle. Each elimination counts over k remaining kittens from the current position onwards and takes away one of the lives of the kitten that the count lands on. When a kitten loses its last life, remove it from the circle and give it endless cuddles and hugs. The circle always tightens to compensate for the eliminated kitten, so the elimination count skips that position in future rounds.

This function should return the elimination order of the kittens, each kitten numbered according to its original position. The main challenge is making this function efficient by maintaining the kitten circle structure so you can step from the current kitten to the next remaining kitten in constant time, regardless of how many kittens have been eliminated between them.

`lives`	k	Expected result
`[4, 1, 3]`	2	`[1, 0, 2]`
`[3, 6, 6, 1]`	3	`[3, 2, 0, 1]`
`[2, 1, 6, 7, 6, 1]`	2	`[1, 5, 2, 4, 3, 0]`
`[5, 2, 9, 5, 1, 8, 10, 10]`	2	`[1, 4, 3, 0, 5, 7, 6, 2]`

22. Maximum repeated suffix

```
def maximum_repeated_suffix(items):
```

This function should find the length of the **maximum repeated suffix** of the given sequence of `items`, that is, the longest suffix that is immediately preceded in that sequence by that very same suffix. This function should not use list slicing to extract the suffix; instead, it should compare in place by looping through indices, which should make it fast even when our automated fuzz tester throws thousands of lists, each containing thousands of elements, at it.

items	Expected result
`[1, 3, 2, 3]`	`0`
`[3, 0, 3, 0]`	`2`
`[2, 0, 1, 3, 1, 1, 3, 1]`	`3`
`[1, 0, 0, 3, 0, 1, 5, 0, 5, 5, 0, 3, 0, 1, 5, 0, 5, 5]`	

23. Count slices with given sum

```
def count_slices_with_sum(items, goal):
```

Given a list of `items`, each item guaranteed to be a positive integer, this function should count the number of separate **slices** in this list whose elements add up to the given `goal`. These slices are allowed to overlap if necessary, and can consist of a singleton element equal to the `goal`.

You could, of course, iterate through all possible starting positions of slices and then, from each starting position, add up elements until the total is greater than or equal to the `goal`. However, to make this algorithm way more efficient, you should note that if you have already tallied up the sum of elements of some particular slice, you can easily compute the sum of elements of that slice without its first element simply by subtracting that first element from the tally.

This approach allows you to compute the answer in a single linear-time loop over the entire list. Use two indices to mark the start and end positions of the current slice, both indices starting at the first element. If the sum of items in the current slice is less than `goal`, advance the end index; otherwise, advance the start index, first incrementing the tally whenever the sum of items equals `goal`. Since each iteration advances at least one index, the algorithm is guaranteed to finish in time linear in the number of elements in the list.

This **two-pointer** approach, using two indices to delimit the sublist in question, is a common technique for solving many list problems in linear rather than quadratic time. In problems like this one, these two indices act as “point man” and “rear guard”, never retreating but always advancing towards the enemy until victory is reached. (In some other problems, for example, the “two summers” problem from course examples, the indices start from opposite ends of the list and always advance towards each other until they meet somewhere in the middle.) As of this writing, the LeetCode category “Two pointers” offers 219 problems amenable to the two-pointer technique.

`items`	`goal`	Expected result
`[2, 1, 1, 1, 2, 1]`	`2`	`4`
`[1, 3, 2, 1, 2, 1, 3, 2, 1, 2]`	`3`	`7`
`[1, 4, 3, 1, 4, 3, 2, 1, 3, 1, 3, 1, 1, 2, 1]`	`8`	`8`
`[3, 4, 4, 1, 1, 3, 1, 4, 2, 3, 4, 1, 3, 5, 5, 4]`	`14`	`7`

24. First sublist whose sum is a multiple of *k*

```
def sublist_sum_k(items, k):
```

Right on the heels of the previous problem, this one swerves in the same direction: determine whether the given list of positive integer items contains at least one nonempty contiguous sublist whose sum is a multiple of *k*. If at least one such sublist exists, return the start position of the earliest such sublist; otherwise, return `None`.

To avoid being a "Shlemiel" who loops through all possible sublists, we need to brush up on elementary number theory to get a working solution that uses a single for-loop over the items, aided by additional data structures. You should keep track of the running **prefix sum** of the elements that the loop has processed so far, and at each element, compute the remainder of the current prefix sum when divided by `k`. Keeping track of all the remainders you have seen during the loop, along with their positions, lets you quickly identify not only such a sublist but also its starting position.

items	k	Expected result
`[1, 10, 15, 23, 35, 22]`	25	1
`[24, 14, 33, 8, 31, 4]`	32	None
`[79, 80, 40, 66, 37, 4, 27, 55, 51, 88]`	133	6
`[87, 187, 71, 125, 4, 175, 81, 110, 66, 124, 28, 66, 87, 193]`	454	None

25. Ones mule, zeros pool

```
def one_zero(n):
```

The original 109 Python Problems collection featured a problem “Sevens rule, zeros drool” that asked the reader to find the smallest integer *m* whose positional base-10 representation is a solid nonempty sequence of sevens followed by a solid (possibly empty) sequence of zeros so that this *m* is divisible by the given *n*. An integer *m* of this restricted form satisfying this requirement always exists. In the same vein, this problem now asks you to find the smallest integer *m* whose positional base-10 representation consists of a solid nonempty sequence of ones followed by a solid (possibly empty) sequence of zeros.

The clever proof to establish the existence of such *m* also immediately gives us a method to construct it. Your function is defined through the **repdigit** integers 1, 11, 111, 1111, 11111, …, noting the remainder left over when each repdigit integer is divided by *n*. If you come upon a repdigit integer divisible by n, that's the one you need. Alternatively, whenever you find two repdigit integers r_1 and r_2 with the same remainder, their difference $r_1 - r_2$ is also divisible by *n*, and is trivially of the required form! Use a dictionary to map the *n* possible remainders to the repdigit numbers that produced them, so that you can quickly recognize that you have already seen the current remainder earlier in the repdigit sequence and subtract those two repdigit numbers.

The famous **pigeonhole principle** guarantees that you must hit the answer after at most *n* steps. (In fact, the equally important **birthday paradox** ensures that you will usually hit the required answer much sooner, on average, roughly around the square root of *n*.)

n	Expected result
1	1
3	111
4	100
11	11
28	11111100
118	11111111111111111111111111111111111111111111111111111111111110

26. The backwards god

```
def count_dogs(grid):
```

You are given a two-dimensional `grid` where each element is one of the three possible characters `'d'`, `'o'` or `'g'`. This grid is a torus, with opposite edges wrapped together; moving off one edge brings you back in from the opposite edge. Your function should count how many times the word `'dog'` appears in this grid. The word can be positioned in any one of the eight directions and can wrap around the edge.

You can solve this with nested loops, but the point of this exercise is to avoid repeating yourself for each of the eight directions and the edge-wrapping logic. In all test cases, both the width and height of the `grid` are at least three.

grid	Expected result

`['ggd',`
` 'ogg',`
` 'ood',`
` 'dgo',`
` 'gdg']`
	`4`

`['dood',`
` 'ogod',`
` 'odoo',`
` 'ggdg']`
	`6`

`['dgdggo',`
` 'gdoood',`
` 'ogdoog',`
` 'gggdoo',`
` 'odoooo',`
` 'doggdg']`
	`12`

27. Klee's measure problem

```
def klee_measure(intervals):
```

A list of `intervals` consists of two-tuples of the form `(start, end)`, each such tuple denoting a closed **interval** on the one-dimensional real number line with the given `start` and `end` locations. The length of that individual interval is then `end - start`. This function should compute the total length of the **union** of these intervals so that any overlapping parts of multiple intervals are included only once in this total length.

This problem can be solved efficiently using the **sweep-line** technique, well known in **computational geometry** problems of all dimensions. Create a sorted list that contains all starting points and ending points of intervals as scalars, each such point paired with either +1 or –1 depending on whether it is a starting or ending point. Loop through the points in this list in their ascending sorted order, keeping a counter to measure the number of intervals that are currently active and overlapping during the journey. When this counter increases from zero to one, remember that point as the starting point of the current interval clump. Once the counter decreases back to zero, add the distance between the current point and the previously stored starting point to the total length.

`intervals`	Expected result
`[(28, 30), (-14, -9), (-4, -2), (-8, -7)]`	`10`
`[(12, 14), (31, 36), (6, 13), (-31, -29), (-21, -14), (27, 29)]`	`24`
`[(34, 39), (-20, -14), (-28, -23), (-16, -13), (27, 30), (-29, -24), (20, 26), (40, 47)]`	`34`

Klee's measure problem is famous because devising a provably optimal algorithm for higher dimensions ($d > 2$) is still an open problem.

28. Poker test of randomness

```
def poker_test(sequence):
```

Statistical tests for pseudorandom number generators calculate various statistical quantities for the sequences they generate and compare them to the quantities a truly random sequence would be expected to have.

One such test looks at all $n - 4$ overlapping sublists of five elements in the given sequence of n elements. It classifies each sublist by the dice poker hand type it corresponds to. This function should return the list of counts of each type of hand in the exact order of the following seven hand types: five of a kind, four of a kind, full house, three of a kind, two pair, one pair, and high card.

sequence	Expected result
`[0, 3, 6, 6, 2, 6, 0, 3, 1, 1, 2, 1, 1, 4]`	`[0, 1, 0, 4, 0, 4, 1]`
`[6, 4, 6, 4, 4, 4, 4, 4, 4, 2, 4, 4, 2, 4, 2, 2, 2, 5, 1]`	`[2, 6, 5, 2, 0, 0, 0]`
`[4, 7, 1, 7, 7, 1, 3, 8, 3, 8, 8, 8, 8, 8, 8, 8, 8, 1, 1, 1, 2, 1]`	`[4, 4, 4, 2, 2, 2, 0]`
`[10, 4, 2, 0, 5, 2, 0, 6, 7, 11, 2, 3, 5, 3, 10, 6, 11, 10, 10, 7, 8, 3, 10, 2, 4, 10, 11]`	`[0, 0, 0, 1, 1, 12, 9]`

29. Three-way partition

```
def three_way_partition(items):
```

A list of `items` consists of three kinds of elements called “small”, “medium”, and “large”. Your task is to tactically **swap** its elements with each other so that all small elements end up being together first in a consecutive block, followed by all medium elements together as another consecutive block, followed by all large elements together as the final consecutive block. The order of elements inside each block is irrelevant, as long as all original elements of the same kind end up together in one consecutive block.

For simplicity, the list contains only elements 0, 1, and 2, denoting the original small, medium, and large elements, respectively. Your function should compute and return the minimum number of swaps required to achieve the required partitioning. The famous Dutch national flag algorithm achieves this minimum number of swaps with a ravishingly beautiful technique often used to prove an algorithm's correctness using loop invariants.

The reader should implement this algorithm from the pseudocode given on the linked Wikipedia page with an additional swap counter whose final value the function then returns in the end. Note that the swap counter should not be incremented in the situation where the algorithm nominally performs a swap of two equal elements. After implementing this algorithm, read through it carefully to appreciate its elegance: it does only what is necessary to achieve the final result, without spinning its wheels at any point during execution.

`items`	Expected result
`[0]`	`0`
`[1, 0, 2]`	`1`
`[0, 1, 0, 2, 1, 2]`	`2`
`[2, 0, 0, 2, 0, 1, 1]`	`5`
`[0, 0, 2, 2, 2, 0, 0, 1, 1, 0, 1]`	`6`

Partitioning algorithms of this kind are used in Quicksort to rearrange elements into separate blocks that can then be recursively sorted independently; these days, even in **parallel** for an even more dramatic speedup. The Dutch national flag partition algorithm produces a good split even when an element is duplicated many times in the list. In contrast, alternative partitioning methods might produce maximally unbalanced partitions, with one element on one side and all the others on the other.

30. Primality buildup

```
def is_prime(n):
```

Many algorithms exist to check whether the given positive integer `n` is **prime**, that is, divisible only by 1 and itself. Here we shall content ourselves with **trial division**, trying potential divisors up to the square root of `n` and concluding that `n` is prime after none of them worked. However, it is sufficient to try only trial divisors that are themselves prime numbers. We shall therefore speed up this process by maintaining a global list of known `primes` that we have found so far, initialized to something like `[2, 3, 5, 7, 11, 13, 17]` to, well, "prime the pump" to generate these trial divisors.

If `n` is less than or equal to the last element of the `primes` list, `is_prime` should check if `n` is a member of the `primes` list using the **binary search** function `bisect_left` in the standard library `bisect` module. Otherwise, the function should iterate through the trial divisors up to the integer square root of `n` (use the `isqrt` function in the `math` module), returning `False` as soon as one working divisor is found, and returning `True` after all the potential divisors have been tried and found wanting.

What if the square root of `n` is large enough for the `primes` not to reach its square root? In addition to the `is_prime` function, write an auxiliary helper function `expand_primes(m)` that keeps appending new prime numbers to the end of the `primes` list until the last prime number is greater than or equal to `m`. If the current last known prime number in the primes list equals *p*, just loop up through the numbers *p* + 2, *p* + 4, *p* + 6, … until you hit a prime number. Use your function `is_prime` to check whether the current candidate is a prime quickly.

Implemented this way, your function should be swift even for numbers up to one trillion, requiring memory to cache the first prime numbers up to one million.

n	Expected result
67	True
8911	False
1234568837	True
1693101241	False

31. Goldbach verification

```
def goldbach(n):
```

Like the twin prime conjecture, the equally famous Goldbach conjecture is another unproven hypothesis stating that every positive integer greater than two can be expressed as the sum of two primes in at least one way. For example, 42 = 5 + 37 and 112 = 3 + 109. (Note again that 1 is not a prime number, but an algebraic **unit**.) This conjecture has been verified for large values of `n`. However, a definitive proof establishing this sweeping claim for every positive even integer still awaits someone to reach mathematical immortality by being the first man or machine to discover it.

For most even numbers, multiple prime pairs add up to them. This function should return the first prime number pair that adds up to `n` in the sense that its first number is as small as possible. Again, use the earlier utility function `is_prime` and the `primes` list that this function maintains to quickly iterate through the candidate prime numbers p needed for this search, returning the first pair $(p, n - p)$ where both p and $n - p$ are prime numbers.

n	Expected result
42	`(5, 37)`
724	`(5, 719)`
1682	`(13, 1669)`
5618	`(37, 5581)`

32. A square and twice a prime

```
def square_and_twice_prime(n):
```

The Goldbach conjecture we met in the previous problem is the most famous conjecture about expressing arbitrary numbers as sums of primes, but similar conjectures exist. A recent tweet by Mike Lawler presents another unsolved conjecture: for any integer $n > 1$, $3 + 8n$ can be expressed as $m^2 + 2p$, where m is a positive integer and p is a prime number. As the results below show, this representation is not necessarily unique. This function should return the sorted list of all values of m that work for the given n in this sense.

n	Expected result
3	[1]
52	[5, 11, 15, 19]
79	[1, 3, 7, 9, 11, 13, 17, 19, 21, 23, 25]
193	[1, 3, 5, 9, 15, 19, 23, 25, 27, 29, 31, 33, 37, 39]
9771391	(a list of 637 possible values of *m*)

33. Pseudo-Smarandache function

```
def pseudo_smarandache(n):
```

The curiously named **pseudo-Smarandache** function Z(*n*) is defined for a positive integer n as the smallest positive integer m such that the *m*-th triangular number, $m(m + 1)/2$, is divisible by *n*. For example, the reader can quickly verify that Z(4) = 7, since of the first seven triangular numbers 1, 3, 6, 10, 15, 21, and 28, only the seventh number 28 is divisible by 4. This function should return the value of *m* that gives that first triangular number.

This problem can, of course, be solved with a **brute-force** search that checks positive integers *m* = 1, 2, 3,... until it finds the first *m* that works for the given *n*. However, for some convenient values of *n*, Z(*n*) can be computed directly with the following rules, without resorting to the brute-force search needed for other values of *n*.

- If *n* is any power of two greater than 1, then $Z(n) = 2n - 1$.
- If *n* is any power of some prime number greater than two, $Z(n) = n - 1$.
- If $n = 2p$ where *p* is an odd prime number, then $Z(n) = p - 1$ whenever $p - 1$ is divisible by 4; otherwise, $Z(n) = p$.

n	Expected result
2	3
5	4
6	3
10	4
17	16
110	44

The pseudo-Smarandache function has mainly significance in recreational mathematics because of its chaotic behaviour and interesting properties, many of which remain open conjectures. For example, $Z(n) \neq n$ for every *n* (as the table above shows, Z(*n*) can be less than *n*), and $Z(n) \neq Z(n + 1)$ for every *n*. The interested reader can search for articles that examine and document both resolved and still open properties of this function. Florentin Smarandache also invented many other integer functions worth examining.

34. Conway's subprime sequence

```
def conway_subprime(a, b):
```

We keep nibbling on the mysteries of the Fibonacci sequence with a simple variation invented by John Horton Conway, because, of course, once again, who else could it have been? (As the famous expression about Isaac Newton goes, you always recognize the lion by its claw.)

Conway's subprime Fibonacci sequence is defined the same way as the ordinary Fibonacci sequence. Still, whenever the next term is composite (that is, not prime), divide it by its smallest prime factor. For example, starting with 1, 1, the next three numbers are 2, 3, and 5, the same as in the original Fibonacci sequence. Still, since the following term, 3 + 5 = 8, is a composite number whose smallest prime factor is 2, this term now becomes 8 / 2 = 4 instead. Use the function `is_prime` implemented in the earlier problem and the `primes` list that this function maintains to quickly iterate through the potential prime divisors of the next term.

Unlike the ordinary Fibonacci sequence, which grows exponentially, the subprime sequence keeps bouncing up and down. It always seems to end in a cycle by reaching a pair of consecutive terms it has encountered earlier. The conjecture that every subprime sequence starting from any two positive integers eventually reaches a finite cycle remains open, similar to the more famous Collatz $3x + 1$ conjecture. Researchers have also checked this conjecture computationally for many starting-value pairs, but a general proof remains out of reach. For our purposes in this problem set, this function should simply return the first repeating pair of terms for the Conway subprime sequence starting from the two initial terms `a` and `b`.

a	b	Expected result
1	4	(11, 9)
3	5	(48, 13)
1	8	(48, 13)
2	17	(24, 7)

35. Rowland's prime pump

```
def rowland_prime_pump(a_start, k):
```

Consider the integer sequence where, after the first element in the first position, 1, the element in each position n is given by the formula a + gcd(a, n), where a is the element in the previous position n – 1, and gcd is the **greatest common divisor** of its parameters, available as the function `gcd` in the `math` module of the Python standard library. The reader can verify that, after this sequence is started from 7, it will continue as 7, 8, 9, 10, 15, 18, 19, 20, 21, 22, 33, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, …

Consecutive values usually increase by 1, but occasionally jump much more. Weird but true: as discussed in the post "Pumping the primes" by Bit Player, assuming certain complex restrictions on the first element of the sequence (these restrictions are trivially satisfied by every prime number greater than six, but also by some sufficiently tricky composite numbers), every time the difference between two consecutive elements is greater than one, this difference itself is guaranteed to be some prime number! This sequence can therefore generate prime numbers in a seemingly random order: not very efficiently, but in an amusing fashion. (Why, you may ask. But why not?)

This function should return the list of the first k distinct prime numbers produced by the Rowland sequence starting from the given `a_start`, listed in the order in which these primes first appeared in the sequence. To make this function efficient, you need to implement the technique described in the linked Bit Player post to skip over the tedious sequences of consecutive ones in the sequence of differences. This technique requires the `is_prime` function from the earlier problem to determine whether $a - n$ is prime, which is necessary for this optimization to apply correctly.

a_start	k	Expected result
7	20	[5, 3, 11, 23, 47, 101, 7, 13, 233, 467, 941, 1889, 3779, 7559, 15131, 53, 30323, 60647, 121403, 242807]
1303	25	[1301, 3, 19, 2621, 7, 29, 5279, 10559, 5, 11, 21143, 42293, 251, 43, 17, 73, 31, 13, 23, 85061, 170123, 89, 340337, 680681, 1361363]
7759	43	[7757, 3, 5, 11, 7, 41, 15581, 31181, 47, 62417, 127, 17, 124991, 19, 53, 29, 31, 250169, 37, 617, 501029, 83, 1002143, 521, 2004809, 4009619, 109, 8019353, 16038707, 727, 32078177, 23, 13, 5431, 151, 179, 761, 64162961, 128325983, 193, 256652189, 73, 317]

36. Lehmer code: encoding

```
def lehmer_code(permutation):
```

In permutation theory, an inversion is a pair of positions $i < j$ where the element in position i is larger than the element in position j. The count of how many inversions the permutation contains measures how "out of order" that permutation is compared to the perfectly sorted permutation whose elements are in ascending order.

The **right inversion count**, also called the Lehmer code of a permutation of n distinct elements, is itself a list of $n - 1$ elements. The element in position i of the Lehmer code counts the number of inversions where that position appears as the first position of the pair $i < j$. Alternatively, the element at position i of the Lehmer code counts how many elements after that position are smaller than the element at position i. Since this count is always zero for the last position of the permutation, the Lehmer code omits this redundant zero.

This function should return the Lehmer code for the given `permutation` of integers 0, ..., $n - 1$.

`perm`	Expected result
`[0]`	`[]`
`[1, 2, 0]`	`[1, 1]`
`[1, 0, 2]`	`[1, 0]`
`[0, 1, 2, 3]`	`[0, 0, 0]`
`[3, 2, 1, 0]`	`[3, 2, 1]`
`[5, 7, 0, 4, 8, 6, 3, 2, 3]`	`[5, 6, 0, 3, 3, 2, 1]`

Since the result of the Lehmer code can always be further interpreted as a factoradic number (see the later problem "Factoradical dudes" in this same collection), this allows the permutations of integers 0, ..., $n - 1$ to be encoded **bijectively** ("one-to-one") to the integers 0, ..., $n! - 1$. Applying the inverse of these two functions in reverse order then decodes these nonnegative integers back into the original permutations.

37. Sum of square roots

```
def square_root_sum(n1, n2):
```

One big thing this instructor tries to do differently than most introductory Python teachers is train students to instinctively avoid floating-point numbers, the same way they would avoid following the voice coming from the sewer telling them they are all just “floating” down there. However, paraphrasing the life story of the reformed mob *caporegime* Michael Franzese on a popular YouTube channel, if you are going to be in the streets, you have to be in the streets the right way. Instead of using the binary floating-point type given by your computer's processor, you should use the `Decimal` type defined in the `decimal` module to perform your floating-point calculations in base ten to the desired arbitrary precision.

Given two lists of positive integers, determine whether the sum of square roots of the numbers in the first list is strictly larger than the sum of square roots of the numbers in the second list. You should perform these computations using the `Decimal` type, increasing the precision until the two sums of square roots are distinct enough.

n1	n2	Expected result
`[6, 9, 13]`	`[5, 10, 12]`	`True`
`[7, 13, 14, 18, 22]`	`[8, 12, 15, 17, 23]`	`False`
`[15, 20, 24, 28, 29]`	`[16, 19, 25, 27, 30]`	`False`
`[14, 17, 20, 23, 24, 28]`	`[15, 16, 21, 22, 25, 27]`	`False`

From a computational complexity perspective, this problem is much harder than it might seem. The intricacies of integer and irrational arithmetic make it hard to determine how many decimal places to compute to be sure which sum is larger. The exact placement of this problem in a specific complexity class is still an open question in computability theory.

38. Night shift

```
def night_shift(data, rules, timeout=10000):
```

Based on repeated **string rewriting**, the esoteric programming language Night shift is simple enough to implement here in its full glory. Yet this language is **Turing-complete**, meaning any computation expressible in any programming language can also be expressed as an equivalent Night shift program! Such languages beautifully illustrate how little machinery is fundamentally needed to achieve all of computability by repeatedly making small, finite, local decisions and modifying linear (but unbounded) `data`.

A program written in Night shift is a list of `rules`, where each rule is a string of the form "left-right", and both left and right are sequences of the digit characters zero and one. A minus sign inside the rule separates the `left` and `right` parts. The `right` part of the rule can also be empty, which is denoted by the special slash character. The rule contains no spaces or characters other than those mentioned above.

The program takes as input a string of zeros and ones. Execution starts by prepending the bit pattern `'000'` to the `data`. For example, the original data `'10110'` would become `'00010110'`. Each computation step first finds the rule whose `left` side occurs leftmost in the current `data`. If the `left` side of several rules occurs in that same leftmost position, use the rule placed the earliest in the `rules` list. If the `left` side of none of the rules matches any part of the `data`, the program terminates and returns as its result the current `data`. Otherwise, replace the matching occurrence of the `left` side of the chosen rule in `data` with the `right` side of that very same rule, keeping the rest of the bits of `data` as they were. Repeat until done. To avoid an infinite loop, we also terminate the program by force after a timeout if it hasn't terminated on its own.

`data`	`rules` (see the `esolangs.org` page)	Expected result
`'10'`	`(remove first bit)`	`'0'`
`'00001'`	`(reverse bits)`	`'10000'`
`'110110'`	(last bit)	`'0'`

39. Infinite self-square integer

```
def self_square(i):
```

In the integers, the equation $x^2 = x$ has only the trivial solutions $x = 0$ and $x = 1$. However, as explained in the YouTube video “Infinite Digits: A Number That Squares to Itself”, this equation has two additional solutions in **infinite integers:** an infinite sequence of digits going left, unlike the familiar decimal part of a real number going right! (If you accept the existence of the latter, why would you not also accept the former? Infinite to the left, infinite to the right, what's the difference?)

Since addition and multiplication of infinite integers use the same column-wise algorithms as finite integers, $x^2 = x$ has two infinite solutions: one starting with 5 in the units place, the other with 6. This function should return the digit in the *i*:th position of the first of these two solutions, where *i* is the power of ten that corresponds to the *i*:th position in that number. The lowest 50 digits of that infinite self-square integer equal

```
...5742342323089610900410661997739225625991821289062 5
```

To find the digit in position *i* of this Eldritch abomination using ordinary integer arithmetic, prepend each digit from 0 to 9 one at a time in front of the finite number that consists of the previously computed digits in positions $i - 1$ down to 0. Square the resulting finite integer, and compare the $(i + 1)$ lowest digits of this square to the original digits. Return the digit whose prepending makes both of those finite numbers equal. (We will simply accept that this works. Source is the classic “Trust me, bro.”) For example, after producing the first two digits 25, only the digit 6 works in the hundreds position, since only $625 \times 625 = 390625$ ends with 625.

For simplicity, the automated tester will call your function starting from $i = 0$ with this parameter increasing by exactly one in each subsequent call, so your function can cache the digit sequence produced so far in the previous calls to speed up the computation of its next digit.

i	Expected result
0	5
2	6
49	5
99	3

As explained in the video, you can easily compute the digits of the other infinite self-square with a 6 in the units place by subtracting each digit from 9. (Also, don’t worry, the infinite self-square integer isn’t real, and it can’t hurt you.)

40. Strahler numbers

```
def strahler(tribute):
```

A river system consists of some number of river segments that occasionally come together in nodes numbered 0, …, *n* – 1, where 0 is the sea that the entire river system flows into. Each node is a tributary to some subset of lower-numbered nodes, so that `tribute[i]` gives the nodes that water flows to from node `i`. No node is ever a tributary to itself or to any higher-numbered node. The river system is not a tree, but a directed acyclic graph where two branches emanating from the same node may eventually meet again at a lower-numbered node.

Given this tributary information, this function should return the list of Strahler numbers for all *n* nodes. As explained on the linked Wikipedia page, the Strahler number of each node is computed recursively from the Strahler numbers of its tributaries, according to the following rules:

- A node with no tributaries has a Strahler number of 1.
- A node whose one tributary has a Strahler number equal to *k*, while all its other tributaries have a Strahler number strictly less than *k*, also has that same Strahler number *k*.
- A node with two or more tributaries with Strahler number *k*, where *k* is the maximum Strahler number among its tributaries, has Strahler number *k* + 1.

Since the nodes and their tributary flows in this problem have already been **topologically sorted**, you can simply compute the Strahler numbers of each node in reverse order from *n* – 1 down to 0.

`tribute`	Expected result
`[[], [0], [0]]`	`[2, 1, 1]`
`[[], [0], [1], [2]]`	`[1, 1, 1, 1]`
`[[], [0], [0], [0, 2], [0, 3, 2], [3, 4]]`	`[3, 1, 2, 2, 1, 1]`
`[[], [0], [0], [1, 0], [2, 3], [3], [5, 4, 3, 0], [0, 6, 4, 5, 1], [4, 6, 0, 2, 3, 5, 1]]`	`[4, 3, 2, 3, 2, 2, 2, 1, 1]`

41. Loopless walk

`def loopless_walk(steps):`

Another neat one from Stack Overflow Code Golf. A series of `steps` visits some letters in the given order. However, we want to shorten this journey by repeatedly performing the following operation as many times as possible: find the first letter *c* that occurs a second time in `steps`, and remove every letter in all positions following the first occurrence of *c*, up to and including the second occurrence. For example, this operation would turn `'baflac'` into `'bac'`. Once this operation cannot be performed because no letter occurs twice in the remaining string, return the remaining `steps`.

This operation is straightforward in theory, but making it efficient for long strings may require avoiding redundant loops over the same prefixes, or building new strings by extracting only a small part near the front.

steps	Expected result
`'zzz'`	`'z'`
`'aasaa'`	`'a'`
`'wczzzv'`	`'wczv'`
`'bhkzmrvfr'`	`'bnkzmr'`
`'llplllilllnell'`	`'l'`

42. Maximum number of blocks during shutdown

```
def max_blocks(permutation):
```

In a row of n lights numbered 0, …, n – 1 where each light is either on or off, a **block** is a maximal sequence of consecutive lamps that are turned on. All lamps are initially turned on, forming exactly one block. These lamps are turned off one at a time, in the order given by the `permutation` passed to the function as a parameter. This function should track the number of blocks formed after each lamp is turned off and return the largest number of blocks encountered during the process.

To efficiently update the block count at each removal, your function should maintain a list of n truth values that indicate which lamps are still turned on. When you turn off the lamp at position i, you can update the block count by looking at the lamps at the neighbouring positions i – 1 and i + 1. Make sure that your logic works correctly at both ends of the row.

`permutation`	Expected result
`[4, 2, 1, 0, 3]`	`2`
`[0, 4, 3, 2, 1]`	`1`
`[6, 2, 1, 3, 5, 0, 4, 7]`	`3`
`[9, 4, 7, 11, 6, 1, 0, 8, 5, 2, 14, 13, 3, 12, 10]`	`6`

43. List of all factors

```
def all_factors(n):
```

This function should return the sorted list of all **factors** of the given positive integer *n*, that is, all positive integers *m* that exactly divide *n*, including *n* itself. For example, the factors of 120 are 1, 2, 3, 4, 5, 6, 8, 10, 12, 15, 20, 24, 30, 40, 60, and 120. Instead of solving this important problem by brute force, looping through all integers from 1 to n / 2 and trying to divide n by each candidate factor, we will use the fundamental theorem of arithmetic: every positive integer has exactly one factorization into its prime factors. Conversely, any two numbers that differ in at least one prime factor must be different. We will therefore first find all the prime factors of *n*,

Use your earlier `is_prime` function to produce the prime factors of *n*, each prime factor listed as many times as it divides *n*. For example, the prime factors of *n* = 120 are `[2, 2, 2, 3, 5]`. Simply loop through the list of candidate primes that your utility function generates, dividing *n* by *p* and appending p to your list of prime factors whenever *p* is a prime factor of *n*. Once *n* becomes 1, you have divided out all its prime factors, and the list is complete.

Next, convert the list of prime factors into a list of lists whose each element list consists of the possible powers of the original prime factor, including the zeroth power equal to 1, for all possible ways that prime factor can be taken to build up a factor of the original *n*. For example, the prime factors list `[2, 2, 2, 3, 5]` would become `[[1, 2, 4, 8], [1, 3], [1, 5]]`.

To finish this off, loop through all possible ways to choose one prime power from each element list, adding the product of your result list of all factors. The `product` function in the standard library `itertools` module should achieve this with flourish. Sort this result list, and Bob's your uncle.

n	Expected result
4	`[1, 2, 4]`
20	`[1, 2, 4, 5, 10, 20]`
179	`[1, 179]`
180	`[1, 2, 3, 4, 5, 6, 9, 10, 12, 15, 18, 20, 30, 36, 45, 60, 90, 180]`
1014	`[1, 2, 3, 6, 13, 26, 39, 78, 169, 338, 507, 1014]`

44. Fourth seat opening

```
def fourth_seat_opening(hand):
```

Sitting at the bridge table in yet another tournament instead of touching grass like a normal person, you see three players pass in front of you and must decide whether to open the bidding or pass out the hand altogether. A handy rule of thumb, the rule of fifteen, also historically called the **Cansino count**, says to add up your raw **high card points** (4 for each ace, 3 for each king, 2 for each queen, and 1 for each jack) and the number of spade cards in your hand regardless of their ranks, and open the bidding if this count adds to 15 or more. The spade suit is special since it's the “boss suit”, and you would like the opponents to afterwards lament ”We missed our spade fit, partner, you should have known to open, herpa derpa derp” whenever you are short in that suit.)

Nothing is certain in life or bridge, but this rule has the highest expected value under uncertainty, over all possible distributions of cards for the other three players consistent with the observed evidence of all three passing out. According to bridge author Larry Cohen, it is also profitable to open in the fourth seat with a pre-emptive bid if you hold a major suit of at least six cards or a minor suit of at least seven cards, and have at least 10 high card points. With 14 high card points, you still open but at one level, since you don't want to miss your game contract when the partner has a good limit raise in your major. (Most serious partnerships don't use two-level pre-empts for minors, but give the 2♣ and 2♦ opening bids a more important artificial meaning.)

Your `hand` is once again encoded the same way as in the example program `cardproblems.py`. This function should determine whether it is profitable to open the given `hand` in the fourth seat.

hand	Expected result
`[('ten', 'hearts'), ('three', 'diamonds'), ('king', 'hearts'), ('seven', 'hearts'), ('six', 'hearts'), ('four', 'diamonds'), ('four', 'hearts'), ('king', 'spades'), ('two', 'spades'), ('eight', 'hearts'), ('seven', 'diamonds'), ('two', 'diamonds'), ('ace', 'diamonds')]`	`True`
`[('ten', 'hearts'), ('five', 'hearts'), ('king', 'diamonds'), ('seven', 'hearts'), ('four', 'diamonds'), ('eight', 'spades'), ('ace', 'spades'), ('four', 'clubs'), ('eight', 'diamonds'), ('eight', 'hearts'), ('seven', 'diamonds'), ('four', 'spades'), ('ten', 'diamonds')]`	`False`

45. Convert a fraction to a simple continued fraction

```
def to_simple_continued_fraction(a, b):
```

Consider a rational number a/b where $0 < a < b$ so that the value of that fraction is between 0 and 1. This fraction can be expressed equivalently as $1/(b/a)$. We can write b/a as $c + (b \bmod a)/a$, where c is the integer part of b/a and is at least 1 since $b > a$. The leftover smaller fraction $(b \bmod a)/a$ can be rewritten similarly. The remaining fraction will eventually be of the form $1/b$, giving us a simple continued fraction, an alternative way to express rational numbers as a list of positive integers. This function should return the simple continued fraction representation of the initial fraction a/b.

For example, consider the fraction 5/17. This can be written as 1/(17/5), which can be written as 1/(3 + 2/5). The inner fraction 2/5 can in turn be written as 1/(5/2) = 1/(2 + 1/2), giving the final complete representation of 5/17 = 1/(3 + 1/(2 + 1/2)). Listing the integer terms in the order that they appear at the levels gives the result list `[3, 2, 2]`.

a	b	Expected result
3	13	`[4, 3]`
5	17	`[3, 2, 2]`
5	61	`[12, 5]`
11	97	`[8, 1, 4, 2]`
20086956	85050054	`[4, 4, 3, 1, 2, 8, 2, 20, 7, 4, 2]`

Simple continued fractions have many curious mathematical properties and practical applications. For example, they help find the rational approximation p/q for a given quantity x, guaranteed to be the best in the sense that no fraction with a smaller denominator than q is closer to x than p/q. The simple continued fraction is always finite for any rational starting numbers and infinite for irrational numbers. This infinite sequence of terms eventually becomes periodic if and only if the original number is a **quadratic irrational**, that is, a solution of some polynomial equation of order two.

46. Convert a simple continued fraction to a fraction

```
def from_simple_continued_fraction(terms):
```

The function is the inverse of the previous problem of converting a rational number to a simple continued fraction in that this function should reverse the conversion and return the original rational number as a Python `Fraction` object. Your function should process the `terms` from end to beginning, adding each term to the accumulated fraction so far and taking the inverse of that result to be the accumulated fraction for the next round of this loop.

`terms`	Expected result (as `Fraction`)
`[4, 3]`	`3/13`
`[4, 6]`	`6/25`
`[3, 2, 2]`	`5/17`
`[6, 1, 8, 6]`	`55/379`
`[13, 6, 11, 6, 2, 4, 1, 13, 7, 6, 15, 2]`	`90669457/1193592058`
`[5, 1, 4, 6, 6, 15, 6, 5, 9, 11, 5, 1, 1]`	`103597999/601519457`

As a side note, in a situation like this, where you are writing two functions that are each other's inverses, it is trivially easy to write a powerful fuzz tester to convince yourself the functions are correct. Generate random inputs for one function, pass the result to the second function, and check that the second function's output matches the original random input. Repeat this a billion times to root out all bugs from your two functions.

47. *Sturm und drangle*

```
def sturm_word(x, y):
```

On a grid paper where all the grid lines are drawn horizontally and vertically between integer points, a line segment is drawn from the origin point (0, 0) to the point (x, y) where x and y are nonnegative integers so that gcd(x, y) = 1. This line segment will intersect some of the horizontal and vertical grid lines along its route from the origin to the point (x, y), but will not go through any corners of the grid until reaching the destination point (x, y). DUCY?

If we denote each crossing of some vertical grid line by `'0'` and each crossing of some horizontal grid line by `'1'`, writing these crossings into a single string creates a finite Sturmian word defined by the slope y/x of the line segment. This function should return the Sturmian word collected during the journey to the endpoint (x, y).

The simplest approach is probably to step in unit steps along one axis, using the slope to advance along the other axis. As much as you may be tempted, please don't use any floating-point numbers in your function, but perform all calculations using exact `Fraction` values. God created the integers; the rest is a work of man.

x	y	Expected result
7	19	`'1101110111011011101110 11'`
19	7	`'00100010001001000100010 0'`
1	5	`'1111'`
6	29	`'11110111110111110111110111110111 1'`
6	31	`'111110111111011111011111011111011111'`

Sturmian words for the grid crossings of the infinitely long lines defined by their slope have interesting mathematical properties that the reader can check out on the linked Wikipedia page.

48. Rational roots of a polynomial

```
def rational_roots(coefficients):
```

The `coefficients` of the polynomial $a_n x^n + a_{n-1} x^{n-1} + \ldots + a_1 x + a_0$ can be given as a tuple of $n + 1$ elements listed in increasing order of powers of x. For example, the polynomial $2x^4 - 3x^3 + x + 9$ would be given as `(9, 1, 0, -3, 2)`, with the missing second-order term having coefficient zero. The rational root theorem says that if p/q is a rational root of a polynomial where p and q are reduced to lowest terms, p must divide a_0, and q must divide a_n. Since only a finite number of p and q satisfy this requirement (don't forget to consider both positive and negative values of p/q), this theorem can find all rational roots of any polynomial effectively and efficiently, regardless of the polynomial's degree.

This function should return all rational roots of the polynomial with the given `coefficients` as `Fraction` objects, listed in ascending sorted order. You should return each root as a `Fraction` object, even if it is an integer. For simplicity, we will also ignore the multiplicity of these roots so that you don't need to implement the polynomial division algorithm. Both the first and the last term of the `coefficients` tuple given to your function are guaranteed to be nonzero, so the polynomial will not have a root at zero.

You probably want to implement Horner's method as a subroutine to evaluate the polynomial with the given `coefficients` at each candidate root p/q. You could, of course, implement some weak-sauce polynomial evaluation method instead, but why settle for second best when the best method is such a concise, beautiful piece of code, and only four lines of Python?

`coefficients`	Expected result
`(-4, -5, -5, -1)`	`[Fraction(-4, 1)]`
`(1, 0, -4)`	`[Fraction(-1, 2), Fraction(1, 2)]`
`(-10, -1, 1, -4, -2, 8, 3)`	`[]`
`(728, 2658, -22225, -1335, 4262, 792)`	`[Fraction(-4, 1), Fraction(-7, 2), Fraction(-13, 99), Fraction(1, 4), Fraction(2, 1)]`

49. Multiple winner election

```
def multiple_winner_election(votes, seats, method):
```

Anglosphere countries tend to use voting systems where each district elects exactly one representative, typically using first-past-the-post plurality to determine the sole winner. Many other countries use some form of party-list proportional voting system where each electoral district has multiple seats to fill. Each voter casts a vote for a particular party, either directly ("closed list") or indirectly ("open list"). The `seats` up for grabs in each district are allocated to the parties in proportion to their `votes` in that district, but so that each party gets a whole integer number of seats.

Given the number of `votes` received by parties numbered from 0, ..., n – 1, this function should compute the distribution of `seats` to these n parties according to these `votes`. Seats are handed out one at a time. Each party has a **priority quantity** that depends on v, the number of votes the party received (which remains constant throughout the process), and s, the number of seats the party has been given so far (which increases throughout the process). The formula for the priority quantity depends on whether you use the D'Hondt, Webster, or Imperiali method. The priority quantity of the party that received v votes and has so far been given s seats is $v/(s + 1)$ in the D'Hondt method, $v/(2s + 1)$ in the Webster method, and $v/(1 + s/2)$ in the Imperiali method. Each seat goes to the party with the highest priority quantity when a seat is awarded; as a result, that party's priority quantity decreases as s increases.

You should perform all these calculations with exact integer arithmetic using the `Fraction` data type, of course. If two parties have the same priority quantity at the time a seat is given, you should break the tie by giving that seat to the party identified with the smaller number.

votes	seats	method	Expected result
`[128, 115, 26, 23]`	`21`	`'dhondt'`	`[10, 8, 2, 1]`
`[128, 115, 26, 23]`	`21`	`'webster'`	`[9, 8, 2, 2]`
`[128, 115, 26, 23]`	`21`	`'imperiali'`	`[10, 9, 1, 1]`
`[40, 37, 35, 15, 12]`	`23`	`'webster'`	`[7, 6, 6, 2, 2]`
`[28, 20, 20, 17, 14]`	`17`	`'dhondt'`	`[5, 4, 3, 3, 2]`

50. Spiral walk

```
def spiral_walk(dx, dy, n):
```

Starting at the origin point (0, 0) of the infinite two-dimensional integer plane, you will make a total of *n* moves. Each move consists of *k* steps in the given **direction vector** (*dx*, *dy*), where initially *k* = 1. After **every two moves**, the step count *k* is increased by one. After each move, rotate the current direction vector counterclockwise by 90 degrees. This rotation is easy without trigonometry or linear algebra: the direction vector (*dx*, *dy*) becomes (–*dy*, *dx*). For example, the vector (3, –2) rotates to (2, 3), which next rotates to become (–3, 2).

With pen and grid paper, the reader can quickly verify, using the starting vector (1, 0) and some small number of steps, that these moves draw a spiral emanating from the origin. This function should return the final position that this spiral walk ends up after *n* moves. As illustrated in the last row of the table below, after a mellow start of small values suitable for debugging, the automated tester will give your function far larger parameter values to chew on. Analogous to the problem "Hippity hoppity, abolish loopity" in the original 109 Python Problems collection, your function should calculate the result in constant time using only integer arithmetic, and not use any loops whatsoever to iterate through the entire spiral path explicitly.

dx	dy	n	Expected result
1	0	2	(1, 1)
-1	1	3	(0, -2)
2	-3	6	(10, -2)
21770	4881	4516163594	(19068371743211, 30090068999249)

Using the initial direction vector (1, 0) and a for-loop nested inside a while-loop, you can write an infinite generator that iterates through all points (x, y) on the infinite two-dimensional integer grid, spiralling away from the origin.

51. Equal sum partition

```
def equal_sum_partition(items):
```

A **partition** of a list is a splitting into nonempty sublists, possibly of different lengths. For example, one possible way to partition the list `[10, 4, 2, 12, 14]` is to split it into `[10, 4]`, `[2, 12]` and `[14]`. In an **equal-sum partition** like this, each piece sums to the same value. Every list can be trivially partitioned into an equal-sum partition by treating the list as a single piece. Still, some lists allow more efficient equal-sum partitions with smaller pieces.

This function should determine the best way to create an equal-sum partition from the given list of positive integer `items` by minimizing the sum of elements in each piece, and return this sum as the final answer. Given a list of n integer `items`, your function should iterate through all possible lengths 1, …, n – 1 of the first piece, and for each such potential first piece, iterate over the remaining elements to determine if they can be partitioned into pieces with that same element sum.

Since all `items` are guaranteed to be positive integers, this test requires no branching choices and can be done with a single linear loop over the elements after the chosen first piece. This loop should maintain a running total of the elements covered so far, initialized to zero. Whenever this running total equals exactly s, the sum of the elements chosen in the first piece, reset the running total to zero; whenever the running total exceeds s, conclude that this attempt at an equal-sum partition failed, and move on to the next longer first piece. Make sure to handle the final piece correctly in this process.

`items`	Expected result
`[7, 2, 8, 1, 5, 4]`	`9`
`[2, 11, 10, 3, 7, 2, 4]`	`13`
`[1, 4, 3, 4, 4, 4, 2, 2, 1, 7, 2, 6, 1, 6, 1]`	`8`
`[5, 2, 10, 1, 2, 13, 1, 16, 1, 4, 6, 7, 16, 1, 17, 9, 5, 2, 1]`	`17`

52. Maximum interval clique

```
def maximum_interval_clique(intervals):
```

A closed **interval** on the real number line is given as a two-tuple (s, e) of its **start** and **end** points, and contains every point x for which $s \leqslant x \leqslant e$. Two intervals **overlap** if they share at least one point; that is, neither ends before the other starts.

Consider the situation of having n such `intervals` whose start and end points have been chosen from integers 0, …, $2n - 1$ so that each one of these $2n$ integers appears exactly once as a start or end point of some interval. Your task is to find the **maximum clique** of these `intervals`, that is, the largest possible subset of intervals so that every pair of intervals in this subset overlaps. Since the clique is not necessarily unique, this function should only return its size.

Unlike general graphs, whose maximal cliques can be enumerated with the Bron–Kerbosch algorithm seen later in this same problem collection, the highly constrained overlap structure of real line intervals allows a far more efficient **sweep-line algorithm** to determine the size of the maximum clique in linear time. We start by noticing that not only does every pair of intervals in the clique contain some common point by definition, but this common point can be chosen to be the same point for all pairs of intervals in that clique! Furthermore, since all interval endpoints are integers from 0 to $2n - 1$, the shared point can be one of these integers. Therefore, you should loop through these candidate integer points from left to right, maintaining a running count of the number of intervals that contain the current point. Use a preprocessing loop through the `intervals` to set up an auxiliary list to tell you what happens at each point so that you can update the running count in constant time at each point during the actual count.

`intervals`	Expected result
`[(4, 5), (1, 2), (0, 3)]`	2
`[(1, 7), (6, 8), (3, 5), (0, 4), (2, 9)]`	4
`[(8, 10), (12, 14), (2, 4), (1, 13), (5, 6), (0, 7), (9, 15), (3, 11)]`	4

53. Maximum overlap intervals

```
def maximum_overlap_intervals(intervals):
```

Same as in the previous problem of finding the maximum interval clique, a closed **interval** on the real number line is given as a two-tuple (*s*, *e*) of its **start** and **end** points, and contains every point *x* for which $s \leqslant x \leqslant e$. Two intervals **overlap** if they have at least one point in common.

Consider again the situation of *n* such `intervals` whose start and end points have been chosen from integers 0, …, 2*n* – 1 so that each one of these 2*n* integers appears exactly once as a start or end point of some interval. This time, your task is to find the intervals that overlap the largest number of other intervals, but unlike in the interval clique problem, those other intervals do not need to overlap each other. Since there can be several such intervals with the same overlap count, your function should return the list of all these intervals, listed in ascending sorted order of their starting points.

Instead of looping through every interval and comparing it to every other interval in an inner loop, you can solve this problem much more efficiently with the classic sweep-line algorithm. Loop through the integers 0, …, 2*n* – 1 while maintaining an **active set** of intervals that contain the current point. At the same time, maintain a **counter dictionary** to track how many intervals overlap each interval. When this loop reaches the point *s* that is the start point of some interval (*s*, *e*), set the count for that interval to the current size of the active set. Next, increment the count by one for each member of the active set, and finally add (*s*, *e*) to the active set. When this loop comes to a point *e* that is the endpoint of some interval (*s*, *e*), simply remove that interval from the active set. A linear preprocessing loop through the `intervals` again allows you to set up an auxiliary list to tell you exactly what to do at each point that the main loop iterates over.

`intervals`	Expected result
`[(0, 5), (1, 6), (2, 3), (4, 7)]`	`[(0, 5), (1, 6)]`
`[(0, 5), (1, 6), (2, 4), (3, 9), (7, 8)]`	`[(3, 9)]`
`[(0, 1), (2, 13), (3, 8), (4, 10), (5, 14), (6, 7), (9, 12), (11, 15)]`	`[(2, 13), (5, 14)]`

Peter Winkler's "Mathematical Puzzles: A Connoisseur's Collection" presents an extremely surprising result: for n intervals constructed randomly from the endpoints 0, …, 2*n* – 1, the probability that one interval overlaps all *n* – 1 others is exactly 2/3 **for every *n* > 1**. Yes, that highly counterintuitive probability is precisely 2/3, even when *n* is up in the gazillions!

54. Split at None

```
def split_at_none(items):
```

The original FizzBuzz problem, once used to screen out utterly hopeless candidates for programming jobs quickly, will soon be almost two decades old and too well known, so our current situation requires more sophisticated tests. Modern problems require modern solutions, as the famous expression goes.

A recent tweet by Hasen Judi offers another interview screening problem that better fits modern scripting languages, yet requires no special techniques beyond the basic imperative programming taught in any introductory programming course. Given a list of `items`, some of which equal `None`, return a list containing, as its elements, the sublists of consecutive `items` split around the `None` elements that serve as the separator marks. Each consecutive block of elements is a separate list in the result. Note the expected correct behaviour whenever these `None` separators are located at the beginning or at the end of the list, and the expected correct behaviour whenever multiple `None` values are consecutive `items`.

items	Expected result
`[-4, 8, None, 5]`	`[[-4, 8], [5]]`
`[None, None, 12, None, 3]`	`[[], [], [12], [3]]`
`[None, None, None, None, 5, None]`	`[[], [], [], [], [5], []]`
`[None, -36, 25, 28, None, -27, -24, 34, -28, 24, 44, 6, 33, 20, None]`	`[[], [-36, 25, 28], [-27, -24, 34, -28, 24, 44, 6, 33, 20], []]`
`[53, 20, -49, 34, None, 13, -43, -14, 53, -6, None, None, None, -26, None, 14, None, None, -11]`	`[[53, 20, -49, 34], [13, -43, -14, 53, -6], [], [], [-26], [14], [], [-11]]`

(Cue the haughty reply tweets of “I have no idea how to solve this totes stupid and meaningless problem that has nothing to do with real work! It's just that those stupid employers don't see what a smart and productive programmer I am in practice, but I just always panic and freeze when I am given simple tests” in three, two, one...)

55. Chunk sorting

```
def chunk_sorting(permutation):
```

Sorting the given random-access sequence of elements in ascending order has been a solved problem during the entire lifetime of most students reading this, and the best algorithms have long been implemented in the standard libraries of all serious programming languages.

The sorting problem remains fruitful for teaching algorithm analysis and related design techniques. Consider the following interesting variation of sorting, given as the problem "Max Chunks To Make Sorted" at LeetCode. Your task is to split the given `permutation` of integers 0, ..., n – 1 into nonempty contiguous chunks so that when each chunk is sorted separately in place (using any one of the standard sorting algorithms; it doesn't matter which one) independently of the other chunks, the entire list becomes sorted. Of course, one chunk spanning the entire `permutation` would always work, but this function should determine the largest number of chunks that can work.

Our automated tester will run your function on 20,000 test cases, most of which contain thousands of elements, so your algorithm needs to operate **in place** on the given `permutation` list and not extract any slices, nor use other inefficient list operations or keep looping back and forth through the list like some Shlemiel. To develop a swift, sure greedy algorithm that runs in a single for-loop through the entire list, never straying or turning back, start by noticing that, of course, the smallest element, 0, must be in the first chunk, but where does that first chunk end? Nope, not necessarily at that element 0, as the third and fifth test cases in the table below illustrate. After that first chunk, where does the second chunk end?

`permutation`	Expected result
`[1, 0, 2]`	2
`[0, 1, 4, 5, 3, 2]`	3
`[3, 0, 2, 4, 1, 6, 5, 7]`	2
`[0, 4, 3, 6, 5, 1, 7, 8, 2]`	2
`[2, 0, 1, 6, 8, 7, 4, 3, 5, 9]`	3
`[1, 2, 0, 3, 8, 5, 10, 4, 7, 6, 11, 9]`	3

56. Merge the two biggest

```
def merge_biggest(items):
```

Your task is to repeatedly perform the following operation for the given list of `items`: remove its two largest elements *a* and *b*, and if their absolute difference *c* = abs(*a* – *b*) is nonzero, insert *c* into the `items`. Rinse and repeat until only one element remains, and return that element as the answer. Should the list become empty after removing the last two elements that were equal, return 0.

For example, if the `items` list initially contains `[7, 1, 5]`, removing the two largest elements, 7 and 5, and inserting their absolute difference, 2, makes the list `[2, 1]`. Repeating the same operation yields [1], and the function terminates, returning 1.

The automated tester will run your function on 10,000 test cases, most of them with thousands of elements. To allow your function to perform this operation efficiently enough to pass the automated tester within the time limit, this would be a good time to turn the `items` list into a **binary heap priority queue** using the functions of the `heapq` module in the Python standard library. The powerful operations `heappop` and `heappush` will then perform the required operations efficiently even for humongous lists... but how to finagle these operations to extract the largest element from the queue instead of the smallest? You need to think up some way to, uh, *negate* the direction of the ordering that is used to organize the elements inside the priority queue.

items	Expected result
`[2, 10, 3]`	5
`[7, 1, 5]`	1
`[7, 11, 11, 1, 15, 11]`	2
`[57, 18, 47, 45, 52, 59, 66, 1, 49, 13, 66, 51, 39, 50, 13, 49, 9, 43, 37, 39, 19, 59]`	1

57. Lehmer code: decoding

```
def lehmer_decode(lehmer):
```

This problem is the inverse of the earlier problem of calculating the Lehmer code encoding for the given permutation. This function should reconstruct and return the original permutation of the nonnegative integers 0,..., n – 1 that produces the given Lehmer code.

This problem is not as straightforward as the mechanistic Lehmer encoding in an earlier problem, but it requires a bit of combinatorial thinking to get started. Note how the first element of the reconstructed permutation must necessarily equal the first element of the Lehmer code. But how would you then construct the remaining required permutation?

lehmer	Expected result
`[1]`	`[1, 0]`
`[1, 0]`	`[1, 0, 2]`
`[1, 1]`	`[1, 2, 0]`
`[2, 0, 1, 0, 1]`	`[2, 0, 3, 1, 5, 4]`
`[2, 3, 0, 2, 2, 0]`	`[2, 4, 0, 5, 6, 1, 3]`
`[7, 1, 1, 4, 1, 0, 1]`	`[7, 1, 2, 6, 3, 0, 5, 4]`

58. Bayesian updating of dice

```
def bayes_dice_update(dice, rolls):
```

You have a box of `dice`, each with a different number of sides used in *Dungeons and Dragons*. Your friend randomly takes one die from the box, rolls it several times out of your view, and then tells you the results. Your task is to calculate, for each die in the box, the exact probability that your friend used that particular die.

Initially, each of the n dice has the same **prior** probability $1/n$ of being the chosen die. However, after seeing the resulting rolls, the posterior probabilities of these dice will usually differ greatly. According to the famous Bayes' theorem, the conditional probability $P(d \mid \texttt{rolls})$ can be written equivalently in the form $P(\texttt{rolls} \mid d)\ P(d)\ /\ P(\texttt{rolls})$. The term $P(d)$ is known to equal $1\ /\ n$, and since the individual rolls are **independent** of each other, the term $P(\texttt{rolls} \mid d)$ can be computed simply as the product of the probabilities of the individual rolls.

To get rid of the problematic term $P(\texttt{rolls})$, note that it is the same unknown constant factor for each die and can therefore be ignored. Therefore, all we need to do is to calculate $P(\texttt{rolls} \mid d)\ P(d)$ for each die, and then **normalize** the resulting probabilities so that they add up to one, seeing that precisely one of these dice must be the one that your friend drew from the box. Of course, you should perform all calculations using exact `Fraction` objects.

dice	rolls	Expected result
`[5, 6]`	`[2]`	`[Fraction(6, 11), Fraction(5, 11)]`
`[3, 4, 5]`	`[2, 2]`	`[Fraction(400, 769), Fraction(225, 769), Fraction(144, 769)]`
`[2, 7, 10, 11]`	`[2, 7, 3]`	`[Fraction(1331000, 2130533)], Fraction(456533, 2130533), Fraction(343000, 2130533)`

59. Haircut

```
def haircut(speed, n):
```

You need a haircut, so go to a popular barbershop with barbers numbered 0, …, *M*, each renting a chair. In this shop, deeply committed to the finest Taylorist principles of standardization and efficiency, the barber numbered `i` will always take `speed[i]` minutes to complete a haircut, regardless of the customer. You arrive at the barbershop in the morning before it opens and take position `n` in the queue of customers already waiting ahead of you. Each customer always walks into the first available chair, regardless of how fast the barber at that chair is. Which barber will eventually service you?

speed	n	Expected result
`[36, 36, 10]`	`7`	`0`
`[28, 16, 51, 36]`	`8`	`3`
`[15, 41, 20, 31, 24]`	`11`	`2`
`[10, 11, 19, 46, 47, 60, 60]`	`22`	`2`

This is the problem “Haircut” from Google Code Jam 2015.

60. Kadane in triplicate

```
def max_three_disjoint_sublists(items):
```

The maximum subarray problem is a classic exercise in algorithmic thinking that asks you to find the contiguous sublist of a given list of integers with the largest sum. (Some of the original list elements are negative so that the answer isn't the trivial “duh, just take that entire list, bro”.) Surprisingly, you can find the maximum subarray with one simple linear loop through the original list using Kadane's algorithm, instead of iterating through a quadratic number of possible start and end positions for all possible sublists.

Initialize a variable *m* to zero, and loop through all the elements *e* in the list. For each list element e, simply update *m* = max(*m* + *e*, *e*). This way, *m* always contains the largest possible sum of any sublist that ends with *e* at that position in the list. Note how we don't care about which preceding elements added up to that maximum sum *m*, only about the value of that maximum sum itself. On the side, keep track of the largest sum that you have seen so far along in the entire list, with the position of that largest sum. Once the loop has terminated, you can reconstruct the sublist using another linear loop going through that sublist in reverse from its stored position.

Now that you know all that, you are hopefully ready to tackle the following problem: find the largest sum that can be formed by adding up the elements of **three** contiguous sublists of your choice from the given list of `items`. All three sublists must contain at least one element, and their positions must be fully **disjoint** so that no position is included in any two of your chosen lists. (Your chosen sublists can be consecutive; there is no need to leave a nonempty gap between them.) Your function should be fast enough to handle thousands of pseudorandom lists with thousands of elements each within the time limit of the automated fuzz tester.

`items`	Expected result
`[-2, -2, -2]`	`-6`
`[2, 3, 4, -5, -5, -5]`	`9`
`[-1, -2, 7, -7, 5, -1, -3, -5]`	`11`
`[-17, -15, 21, 44, -20, 40, -21, -11, -15, 31, -17, -21, -11, 25, -4, -4, 47, -11, 44, -17, -9, -8, -19, -22]`	`213`

61. Yellowstone sequence

```
def yellowstone_sequence(n):
```

This pleasantly chaotic **infinite permutation** of all positive integers comes from the Numberphile video “The Yellowstone Permutation”. This sequence is based on a common structure: give the first couple of values, then choose the next element as the smallest unseen positive integer that satisfies the simple local constraint. This local requirement for the Yellowstone permutation is that if *b* is the most recent previous element and *a* is the second most recent one, the next element *c* of the sequence **must** be relatively prime to *b* (that is, it must be that gcd(*b, c*) = 1), and it **may not** be relatively prime to *c* (that is, gcd(*a*, *c*) > 1).

Setting the first three elements of the sequence to be simply 1, 2, and 3, the patient reader can verify that this sequence starts as

1, 2, 3, 4, 9, 8, 15, 14, 5, 6, 25, 12, 35, 16, 7, 10, 21, 20, 27, 22, 39, 11, 13, 33, 26, 45, 28, 51, …

Using 0-based position indexing since we pretend to be proper computer scientists, this function should return the element in position *n* of the Yellowstone sequence. The automated tester will give your function the consecutive values of `n` from 0 up to 9999 in this order, so that you can store the numbers seen so far in a global set that maintains its contents between the function calls.

`n`	Expected result
`0`	`1`
`4`	`9`
`9`	`6`
`99`	`165`
`999`	`920`
`9999`	`11649`

As with many other sequences of this nature, it is not self-evident that this infinite sequence is a permutation of the positive integers, that is, it contains every positive integer exactly once without leaving any gaps. The Numberphile video linked above contains a lovely proof of this fact, as presented by Neil Sloane himself.

62. Largest rectangle in a histogram

```
def largest_rectangle(histogram):
```

This problem is “Largest Rectangle in a Histogram” in Peking University Online Judge. You are given a `histogram` that specifies the heights of the bars for positions 0, …, n – 1. This function should find the largest axis-aligned rectangle that fits inside this histogram, and return its area.

The difficulty in this seemingly simple problem is devising an efficient algorithm that works even for thousands of pseudorandom problem instances with thousands of elements. There are many ways to achieve this efficiently. The simplest approach is to use a Python list as a stack for possible starting x-coordinates of rectangles, removing them when you encounter a bar low enough to end that rectangle.

`histogram`	Expected result
`[3, 9, 8, 12]`	`24`
`[3, 1, 4, 2, 1]`	`5`
`[14, 13, 13, 2, 8, 3]`	`39`
`[21, 21, 18, 2, 1, 4, 5, 4, 5, 7, 7, 4]`	`54`

63. Frobenius nuggets

```
def frobenius(coins):
```

The famous Frobenius coin problem, sometimes called the **McNugget problem**, asks for the largest positive integer that cannot be written as a sum of available coins, where you can use each coin denomination as many times as needed. For example, 43 is the smallest integer that cannot be expressed as a sum of `[6, 9, 20]`, whereas all integers less than 43 can be expressed in this manner. This function should determine the Frobenius number for the given list of `coins`. For simplicity, you may assume that the `coins` are **coprime** (which guarantees a finite solution), and the `coins` are given in ascending sorted order. So that the answer isn't trivially zero, all coins are greater than 1.

This problem is best solved with **dynamic programming**. Loop through the integer values from k = 2 upwards, filling in a list of truth values telling whether each k can be expressed as a sum of the available `coins`. This is possible precisely when at least one of the integers $k - c$ can be expressed this way, where c runs over the available `coins`, which you can check easily with a simple nested loop. Once you have found a run of m consecutive `True` elements, where m is the smallest coin denomination available, the integer that precedes that run is the desired Frobenius number.

As the fourth example in the table below shows, the Frobenius number can be smaller than the largest available coin.

coins	Expected result
[6, 9, 20]	43
[29, 37]	1007
[13, 67, 97]	412
[3, 23, 47, 53]	43
[11, 29, 41, 67]	112

As an aside, 1 is not considered a prime number. While it's often explained through the lens of unique prime factorization, the deeper reason is that 1 is what mathematicians call a unit: a number that “divides” everything. This concept generalizes beyond integers to more abstract algebraic structures that the reader will encounter in later studies.

64. Postage stamp problem

```
def postage_stamp(stamps, k):
```

The Postage stamp problem is very similar to the previous Frobenius problem, and can be solved with a similar general approach. You are given an unlimited supply of `stamps` of given face values. All stamps are the same size, and the postcard can fit at most *k* stamps. What is the smallest total postage that cannot be placed on the postcard under these restrictions? (The smallest stamp is always worth one cent, so any amount of postage is theoretically possible if we ignore the limit *k*.)

The simplest way to solve this problem is to use a dynamic programming approach similar to the one we used in the Frobenius problem. Fill in a list of values $M[v]$ of the number of stamps required to achieve a total of *v.* Assuming that you have filled this list up to the value $v - 1$, the value $M[v]$ equals one plus the minimum of the values $M[v - s]$, where *s* loops through the given `stamps`. Once you reach a position *v* where $M[v] = k + 1$, the current position *v* is the final answer.

stamps	k	Expected result
[1, 3, 7, 8]	6	49
[1, 3, 10, 13]	6	61
[1, 8, 12, 15, 19, 21]	12	245
[1, 11, 16, 21, 25, 27, 30]	13	383
[1, 2, 14, 27, 34, 39, 49, 50, 53, 68]	5	11

65. Knaves of the round table

```
def knaves_of_round_table(answers):
```

Some knights and knaves are gathered cyclically around King Arthur's round table. These men are numbered 0, …, n – 1 by position. As we learned from Raymond Smullyan, knights always tell the truth and knaves always lie. Unfortunately, we cannot tell which men are knights and which are knaves. (If asked directly which kind of man he is, each man will just claim to be a knight; not very helpful to us.)

To start resolving this mess, we ask each man how many of the two men sitting immediately to his left and right are knights, and collect the `answers`. These `answers` do not always uniquely pinpoint the positions of the knights and knaves, but they do let us identify all possible assignments consistent with them. For example, there are two possible solutions consistent with the three men's answers `[2, 2, 2]`: either all three men are knights telling the truth, or all three are knaves with their pants on fire.

This function should return the list of possible counts of how many knights are seated at the round table. Note that the function does not return the possible positions of these knights, but the possible total number of knights. The automated tester will only give your function `answers` for which there exists at least one assignment of knights and knaves consistent with those `answers`.

This function can recursively fill out all possible assignments of these men to knights and knaves from left to right, backtracking from the partial assignment built so far as soon as it hits a local contradiction. Most branches will be pruned immediately by a local contradiction, making this recursive search fast even if hundreds of men were seated around King Arthur's table.

answers	Expected result
`[1, 2, 2]`	`[0]`
`[0, 2, 0]`	`[1]`
`[2, 1, 1, 1]`	`[0, 3]`
`[2, 2, 2, 2, 2, 2]`	`[0, 6]`
`[1, 2, 2, 1, 2, 1, 2]`	`[0, 3, 4]`
`[0, 1, 2, 2, 2, 2, 2, 2, 1]`	`[8]`

66. Six of this, half a dozen of the other

```
def assign_sides(costs):
```

You are given 2*n* items, and you need to assign exactly *n* of these items of your choice to this side, and the remaining *n* items to the other side. Each item is represented as a two-tuple `(this, other)` that gives the cost of assigning that item to this side and to the other side, respectively. This function should compute the optimal assignment of these 2*n* items to this side and the other side to minimize the total overall cost of the assignment.

This problem may initially seem like a backtracking recursion where each item can be assigned either way, terminating at the base case where neither side has *n* items assigned. However, this would be serious overkill; clever preprocessing of these costs allows you to compute the optimal assignment in a single linear pass over the costs. The key insight is that you must assign every item somewhere, and one dollar saved is one dollar saved, no matter where it comes from.

costs	Expected result
`[(4, 6), (1, 3), (3, 2), (3, 5), (4, 2), (3, 1)]`	13
`[(1, 1), (8, 6), (7, 9), (4, 6), (2, 9), (3, 2), (8, 3), (6, 4), (4, 3), (3, 5)]`	35
`[(5, 9), (5, 1), (4, 7), (7, 11), (10, 7), (11, 11), (11, 8), (10, 9), (5, 9), (5, 9), (2, 11), (8, 8), (11, 3), (7, 1)]`	76

67. Factoradical dudes

```
def factoradic_base(n):
```

Several problems in our original two Python problem collections have shown how to represent integers in positional number systems in all sorts of weird bases. Instead of the familiar positive integer bases ten and two, these bases could be negative, rational, or even complex numbers whose powers, multiplied by the coefficient of that position, add up to that integer.

In this problem, we look at an interesting positional number system that uses a **mixed base** where the base changes with each position. A variable base might initially seem weird. However, you already use such a system every day when expressing time: seconds and minutes are given in base 60, while hours are expressed in either base 12 or 24, depending on which side of the pond you live on. Imperial measurements for weights and lengths also use mixed bases, with each unit expressed as a multiple of the next lower unit.

The factoradic number system uses the factorials 1, 2, 6, 24, 120, ... as variable bases. If the positions are numbered starting from 1, the base in position k equals $k!$, the product of the first k positive integers. The possible coefficients for the position k are then 0, ..., k. If the coefficient at position k were $k + 1$, the resulting term $(k + 1)k!$ would equal $(k + 1)!$ and could therefore be carried over to the next position, same as in any other positional number system.

This function should return the factoradic representation of the given positive integer n as the list of its factoradic coefficients so that the coefficient in the position $k - 1$ of the result list gives the coefficient of the factorial $k!$ in the representation. The automated tester will give your function some pretty humongous numbers to convert. Still, since factorials grow so fast, your function should return the coefficient list quickly, even for numbers with hundreds of digits.

n	Expected result
1	[1]
11	[1, 2, 1]
96	[0, 0, 0, 4]
1781	[1, 2, 0, 4, 2, 2]
4146434844171	[1, 1, 0, 2, 3, 1, 6, 7, 2, 10, 4, 11, 7, 2, 3]

Factoradic numbers can uniquely encode arbitrary permutations of distinct elements into positive integers using the Lehmer coding from earlier problems.

68. Tarjan's strongly connected components

```
def tarjans_scc(edges):
```

The basic graph search algorithms, breadth-first search and depth-first search, can be augmented to compute other graph properties. Consider the problem of finding the strongly connected components of a given directed graph: maximal subsets of nodes such that you can get from any node in the component to any other node in the same component. Still, once you leave the component, you cannot return. (If there were such a way back, the node that you exited to would, by definition, be part of that same strongly connected component.) The strongly connected components of a graph partition its nodes, with each node belonging to exactly one strongly connected component. If nothing else, each node by itself is a singleton component.

Tarjan's strongly connected components algorithm is an ingenious application of depth-first search that enumerates all strongly connected components as a byproduct of a single DFS. Your task here is to translate the pseudocode from the Wikipedia page (or any other source where fine algorithms are sold) into Python. Python's basic list type works well as the `index`, `lowlink`, and `on_stack` bookkeeper, and as the stack from which to pop the nodes of the strongly connected component represented by the current node whose index and lowlink end up being equal.

Your function is given a directed graph whose nodes are integers 0, …, $n - 1$, and the list of `edges` emanating from each node. The function should return the list of strongly connected components so that the nodes of each component are listed in ascending sorted order, and the components themselves are listed in ascending order of their lowest-numbered nodes.

`edges`	Expected result
`[[1, 4, 3, 2], [3, 4, 0], [4, 1, 0, 3], [2], [0, 3, 2, 1]]`	`[[0, 1, 2, 3, 4]]`
`[[4, 3], [2, 5, 4], [4, 1, 0, 3, 5], [4, 0], [], [1, 2]]`	`[[0, 3], [1, 2, 5], [4]]`
`[[2, 11, 10, 3, 12], [0, 3, 12, 2, 11, 4], [1, 0, 12], [4, 1, 6], [7], [4, 8], [8, 4], [4, 5], [6], [8, 10, 6, 12], [9, 12, 11, 7], [10], [1, 10, 0, 11]]`	`[[0, 1, 2, 3, 9, 10, 11, 12], [4, 5, 6, 7, 8]]`

As a nostalgic trip down memory lane, your author spent two days understanding and implementing this algorithm in C for his Master's thesis back in 1995, whereas now the same task takes him about fifteen minutes.

69. Copeland harder

```
def copeland_method(ballots):
```

Consider an election in which multiple candidates compete to be the single winner in that district. First-past-the-post plurality voting, used in most Anglosphere elections, has simplicity and tradition on its side, but it lacks many desirable properties of fair election systems.

In Copeland's method, instead of voting for a single preferred candidate, each voter ranks all candidates in descending order of preference on the ballot. To determine the winner, each candidate faces a one-on-one match against every other candidate. In this one-on-one match, voters ignore all other candidates and choose the candidate they prefer between the two, based only on those two candidates on each ballot. The candidate who wins the one-on-one match gets one matchpoint for this win. Should the candidates tie, both receive half a matchpoint.

Note that the magnitude of winning a single match does not affect the matchpoint gain; beating the opponent by one vote gives that candidate the same one matchpoint as winning by a million votes. It also doesn't matter how much each voter prefers one candidate over the other, since only ordinal preference matters, regardless of the cardinal magnitude of that preference.

The overall election winner is the candidate with the most matchpoints. However, unless voter preferences determine a unique Condorcet winner, the mechanism can tie. This function should return the sorted list of winners in the election among candidates 0, ..., $n - 1$, where each ballot is a permutation of these candidates indicating the voter's preferences.

ballots	Expected result
`[[1, 0, 2], [2, 0, 1], [2, 1, 0], [0, 2, 1], [1, 2, 0]]`	`[2]`
`[[1, 2, 0, 3], [3, 2, 0, 1], [2, 3, 0, 1], [1, 3, 0, 2], [3, 1, 0, 2], [2, 1, 0, 3]]`	`[1, 2, 3]`
`[[1, 0, 2, 3], [3, 0, 2, 1], [0, 3, 2, 1], [3, 0, 2, 1], [0, 3, 2, 1], [1, 3, 2, 0]]`	`[0, 3]`

70. Instant-runoff voting

```
def instant_runoff(ballots):
```

Following the previous Copeland method problem, consider an election again where multiple candidates compete for the single winner of that district. As with the Copeland method, instant runoff voting has voters rank all candidates in order of preference on the ballot. The difference is that pairwise matches do not determine the winner, but by repeatedly eliminating the least favoured candidate in a plurality vote until a candidate wins a strict majority.

Initially, each ballot goes to the candidate who was that voter's first preference. If no candidate gets a majority of votes, eliminate the candidate with the fewest ballots and redistribute their ballots to the remaining candidates, giving each ballot to the candidate who was that voter's second preference. Elimination continues until one remaining candidate has a strict majority of votes (*more* than half) and is declared the winner.

This function should determine the winner of candidates numbered 0, ..., $n - 1$ in the instant runoff election, when each ballot is a permutation of these candidates listed in the preference order of the voter casting that ballot. To make the results unambiguous for the automated fuzz tester, if several remaining candidates have the same smallest number of ballots in the current round, eliminate the highest-numbered such candidate.

To implement this function correctly, note that a candidate who has been eliminated in any previous round cannot come back to life by receiving ballots from candidates eliminated in the current round. However, each of their ballots passes to that voter's next preferred candidate who is still alive and kicking in the election.

ballots	Expected result
`[[1, 0, 2], [2, 0, 1], [2, 1, 0], [0, 2, 1], [1, 2, 0]]`	2
`[[1, 0, 2], [0, 1, 2], [1, 0, 2], [2, 0, 1], [0, 2, 1]]`	1
`[[1, 4, 2, 3, 0], [3, 4, 2, 1, 0], [2, 4, 3, 1, 0], [3, 4, 2, 1, 0], [2, 4, 3, 1, 0], [4, 2, 3, 1, 0], [3, 2, 4, 1, 0], [4, 2, 3, 1, 0]]`	4
`[[1, 0, 2, 3, 4, 5], [5, 0, 2, 3, 4, 1], [0, 5, 2, 3, 4, 1], [1, 2, 5, 0, 4, 3], [0, 2, 5, 1, 4, 3], [3, 5, 4, 0, 2, 1], [4, 5, 3, 0, 2, 1], [5, 4, 3, 0, 2, 1], [5, 2, 0, 3, 4, 1]]`	5

71. Cousin explainer

```
def cousin_explainer(a, b):
```

Women in a matrilineal family tree are numbered so that Eve at the root has the number 1, and the woman whose number is *i* has exactly two daughters, numbered 2*i* and 2*i* + 1. For example, the two daughters of woman 7 are the women numbered 14 and 15. In this scheme, the mother of woman *i* is given by the formula *i* / 2, rounded down to an integer for odd *i*. As you can see, this formula gives the mother's number 7 for both of her daughters, 14 and 15. (This is also exactly how a binary tree structure is imposed on a list when treating it as a **binary heap**, as implemented in the `heapq` standard library module.)

Given two women numbered `a` and `b`, this function should return in English what woman `b` is to woman `a`. For simplicity, assume that all the men these women procreated were genetically distinct enough that the familial relationship between any two women in this problem is determined solely by this matrilineal tree.

To determine the relationship between women `a` and `b`, trace the parent lineages from both women to their **nearest common ancestor**, keeping track of the number of steps from each woman to that ancestor. These two counts determine the relationship, as illustrated in the tweet "The cousin explainer". For example, if one of these step counts is zero, that woman is a direct ancestor of the other one. If one of the step counts is one, that woman is the (some number of `great`) aunt of her (some number of `great`) niece. Otherwise, these two women are cousins to the degree determined by the smaller of the two step counts to their common ancestor. The count of **removals** is determined by the difference between these two step counts. For this count of removals, use `'once'`, `'twice'` and `'thrice'` for the first three, after which use `'four times'`, and so on. Do not use hyphens; say "great grandniece" instead of "great-grandniece".

a	b	Expected result
1	3	`'daughter'`
3	5	`'niece'`
5	18	`'grandmother'`
9	7	`'first cousin once removed'`
3	18	`'great great grandniece'`
344	28	`'third cousin four times removed'`
612	5	`'great great great great great grandaunt'`
1133	3668	`'ninth cousin once removed'`

72. One-dimensional random walk

```
def random_walk(moves, n, positions):
```

Starting from the origin point zero of the one-dimensional integer line, you take a **random walk** of exactly `n` steps so that, independently of the previous steps, each step either moves one step to the left, stays put in the current position, or moves one step to the right according to the probabilities given in the list `moves=[left, stay, right]`, these three transition probabilities always adding up to one. This function should compute and return, as a batch job, the exact probability (as a `Fraction` object) of this random walk finishing in each of the given `positions`.

Combinatorics mavens could probably come up with a closed-form formula for these probabilities. Still, here we can solve them with a recursive formula in terms of the number of steps n. The base case with $n = 0$ is trivial. After that, the probability $P(n, k)$ of ending in position k after taking n steps can be computed from the probabilities $P(n - 1, k - 1)$, $P(n - 1, k)$ and $P(n - 1, k + 1)$ for ending up after $n - 1$ steps in a position where you can move to position k in a single step, and adding up these probabilities weighted by their corresponding transition probabilities to position k.

You can either implement this recursion as a nested recursive function using the `@lru_cache` to memoize the overlapping subproblem results to prevent exponential blowup of running time, or implement it with **dynamic programming** by computing the table of probabilities directly. You also only need to maintain the current and the previous row of this table, instead of lugging around the entire two-dimensional structure.

moves	n	pos	Expected result
`[Fraction(1, 2), Fraction(1, 10), Fraction(2, 5)]`	`2`	`[0]`	`[Fraction(41, 100)]`
`[Fraction(2, 3), Fraction(1, 5), Fraction(2, 15)]`	`4`	`[-4, -2]`	`[Fraction(16, 81), Fraction(536, 2025)]`
`[Fraction(1, 2), Fraction(1, 18), Fraction(4, 9)]`	`7`	`[1, 4]`	`[Fraction(14159887, 76527504), Fraction(97888, 4782969)]`

73. Knight watch

```
def knight_watch(n, x, y, k, pillars):
```

One of the blind chess knights from the problem “Sneaking“ in the earlier Additional Python problems collection is the lone guardian of an *n*-by-*n* chessboard outside of which nothing exists except impenetrable stone walls that continue solid to infinity in every possible direction. Some stone `pillars` also stand at certain locations on the board. This long-forgotten knight can observe the square he is on and all squares one knight's move away. The knight cannot move inside a pillar (but neither can the king), but otherwise, these pillars don't hinder his movement or sensing. Even in his old days, this knight still has his movement powers, so he can move and see over a pillar, just not into it.

Waking up at the coordinates `(x, y)`, the weary old knight can only make `k` moves before going to sleep again. A loyal servant of his lost empire, long since ground to dust, the knight still wants to guard his board as effectively as possible. Even though he can't necessarily check the entire board, he wants to choose a sequence of k moves that lets him observe the most squares along the way, to catch any wayward enemy kings that might still be sneaking around after millennia of abandonment. Since this optimal sequence isn't necessarily unique, this function should return the maximum number of squares the knight can observe in k moves before he goes back to sleep, only to repeat the same thing the next day.

n	x	y	k	pillars	Expected result
8	4	4	0	[]	9
8	4	4	1	[]	16
4	0	3	2	[(0, 0), (3, 1), (3, 2), (3, 0), (1, 0), (1, 1)]	5
4	2	0	2	[(3, 2), (1, 1), (0, 0)]	8
6	0	3	3	[(5, 3), (2, 3), (4, 2), (5, 2), (5, 5), (1, 4)]	19

Enjoyers of chessboard combinatorics are surely familiar with the knight's tour problem, where a chess knight must visit each square exactly once and return to its starting square. An interesting variation asks for the shortest path for our blind knight to fully observe the entire 8-by-8 chessboard.

74. Longest Fibonacci-like subsequence

```
def longest_fib_subsequence(items):
```

A sequence of positive integers is said to be **Fibonacci-like** if, after the first two elements, each element equals the sum of the two immediately preceding elements. (By this definition, every sequence of two elements is trivially Fibonacci-like.) This function should find the longest Fibonacci-like subsequence of the given list of positive integer `items`. Since this subsequence is not necessarily unique, the function should return only the length of that subsequence. Also, as you can see in the second and third example test cases in the table below, the second element of the subsequence does not need to be strictly larger than the first element.

The main difficulty of this problem is in making this function fast enough to pass the automated fuzz tester within the time limit. The trivial $O(n^3)$ algorithm of iterating through all possible positions of the first two elements and finding the longest Fibonacci-like sequence emanating from each such element pair in a single linear pass through the remaining elements is too slow to pass the fuzz test suite within the time limit, so you might want to preprocess the sequence to optimize the subsequence generation, or think up some other algorithmic optimization that is applicable here.

items	Expected result
[7, 4, 10, 6, 4]	2
[6, 4, 9, 3, 7, 13]	3
[6, 3, 6, 12, 15, 12, 18, 30, 27]	5
[6, 11, 17, 23, 28, 45, 73, 118, 191, 309, 500, 326]	10

75. Maximize exam confusion

```
def maximal_confusion(answer_key, k):
```

This deceptively simple problem is LeetCode problem 2024, “Maximize the Confusion of an Exam”. Too dumb to create new original exams himself, this lazy teacher has inherited a multiple-choice exam whose answer_key consists of characters 'T' and 'F' indicating whether the correct answer to each question is `True` or `False`. Being generally unfit for his teaching position and an insecure impostor who knows that his students, who paid hard-earned money to take the course, will soon notice his incompetence unless their minds are distracted elsewhere, he decides to play sadistic mind games with his students and change at most *k* questions to opposite answers to maximize the longest run of consecutive questions with the same answer, either all `True` or all `False`, to gaslight the exam takers into questioning their sanity.

Of course, even this *mamaluke* will soon realize that all *k* changes should go in the same direction (either change some `True` answers to `False`, or change some `False` answers to `True`), and soon after that, realize there is no point in leaving gaps between these changes. Still, all changes should occur consecutively within a subsequence of questions with the same answer. After these flashes of insight, perhaps you can hold your nose and help him to achieve this maximal confusion.

Your function should return the length of the longest run achievable by at most *k* changes. Despite the despicable premise, you can polish this turd enough to reveal the diamond underneath. Try to solve this problem in a single for-loop with O(1) extra memory, for a solution so ravishingly beautiful in its simplicity that any student working through it and proving it correct will instantly level up in their algorithmic and coding problem skills.

answer_key	k	Expected result
'FTFFFF'	3	6
'TFFTTFFTT'	2	6
'TTFFFTFFTFT'	3	9
'TFFFFTTTTTFFFF'	3	8

76. Burrows–Wheeler transform

```
def burrows_wheeler_encode(text, positions):
```

The Burrows–Wheeler transform is a famous algorithm that converts any text string into a string of equal length, permuting the original characters so compression algorithms can compress it much better than the original string. Of course, after decompressing to recover the transformed string, this transform can be reversed effectively and efficiently to restore the original `text`, which is a topic for a follow-up question left hanging here.

This function should compute the Burrows–Wheeler transform of the given `text` string. However, instead of returning the entire transformed string, you should only return the characters in the given `positions` of the result as **proof of work** of having computed this transformation, while keeping the result small for our automated fuzz tester and the `expected_answers` file.

The educational point of this exercise is to take the inefficient Python implementation that explicitly constructs all cyclic rotations as the starting point, and rewrite it to operate **in place** so that only one copy of the `text` is kept in memory at all times. The fuzz tester will give your function long strings composed of random words from the `words_sorted.txt` file, so efficient operation of your function here is a must. Each `text` is also guaranteed to end with the terminator `'$'`.

Your function should create the list of all position indices in the text and sort this list with a **custom comparator** function that, given two positions `i` and `j`, compares the two cyclic rotations of the `text` starting from those positions in lexicographical order, traversing the characters in lockstep from both positions and comparing them pairwise until a mismatch is found. Use the `cmp_to_key` function of the Python `functools` standard library module to pass your custom comparator function to the `sort` method, and then just extract the last characters of the cyclic substrings starting from the indices in the `positions` given.

`text`	`positions`	Expected result
`'a$'`	`range(2)`	`'a$'`
`'abc$'`	`range(4)`	`'c$ab'`
`'abab$'`	`range(5)`	`'bb$aa'`
`'baab$'`	`range(5)`	`'bbaa$'`
`'pilloriesafterpartlepidopterous$'`	`[10, 12, 17, 23, 27]`	`'rtree'`

77. Bron–Kerbosch maximal clique enumeration

```
def maximal_cliques(edges):
```

The pairwise friendship relation of a group of *n* people is expressed as a graph, where `edges[u]` is the list of friends of the person *u*. Friendship is assumed to be symmetric in that *v* is a friend of *u* whenever *u* is a friend of *v*, so this graph is effectively undirected. Some people are loners with no friends, so their friend list is empty.

In graph theory, a **clique** is a set of nodes that are all pairwise friends. A clique is said to be **maximal** if no node can be added to it without breaking the defining clique property of universal pairwise friendship. This function should return the list of all maximal cliques of three or more nodes in the graph with the given `edges`. The nodes of each maximal clique should be listed in ascending order, and the cliques themselves should be listed in descending order of size, with cliques of equal size listed in ascending lexicographic order. Unlike in the earlier Tarjan's strongly connected components problem, maximal cliques do not form an equivalence partition of the nodes in that graph. Still, a social person can belong to multiple separate maximal cliques.

You should implement the Bron–Kerbosch maximal clique enumeration algorithm as a nested function inside this function. Maintain a list of maximal cliques outside the recursion, and append the maximal cliques to this list as your recursion finds them. The pseudocode of the Bron–Kerbosch algorithm should translate neatly into Python and its built-in operations on sets. To ensure your function passes our pseudorandom fuzz test suite in a reasonable time, implement the version of the Bron–Kerbosch algorithm that uses pivot nodes, choosing a good pivot each time to prune redundant branches of the search ruthlessly.

`edges`	Expected result
`[[1, 3, 2, 5], [0, 2, 4, 3], [1, 0], [0, 1, 5], [1], [0, 3]]`	`[[0, 1, 2], [0, 1, 3], [0, 3, 5]]`
`[[1, 4], [0, 2, 5, 4, 7, 3], [1, 7, 3], [2, 1, 5], [0, 1, 7], [1, 7, 3], [], [2, 5, 1, 4]]`	`[[0, 1, 4], [1, 2, 3], [1, 2, 7], [1, 3, 5], [1, 4, 7], [1, 5, 7]]`
`[[1, 3, 6, 2, 5], [0, 2, 5, 3], [1, 3, 0, 5], [0, 2, 6, 1, 5], [], [1, 0, 2, 3], [0, 3]]`	`[[0, 1, 2, 3, 5], [0, 3, 6]]`

78. Queue jockeys

```
def queue_jockeys(permutation, moves):
```

A simulated system consists of two queues. The first queue initially contains the integers 0, …, n – 1 as listed from front to back in the given `permutation`, whereas the second queue is initially empty. A series of `moves`, each naming one of the stored elements, is executed in order so that each move removes that element from whichever queue it is currently in and places it at the front of the other queue. This function should return a two-tuple giving the contents of the first and second queues as lists after all these `moves` have been executed.

Of course, at this point in your studies, this function should not be hard to write using two Python lists to store the contents of these queues, and using simple Python list slicing and dicing operations to perform each move. Alas, after the smooth start of small inputs suitable for debugging, the automated tester will give your function inputs where n is in the thousands, and the number of `moves` is in tens of thousands, making sure that any solutions based on list slicing and dicing will not pass the automated tests within the set time limit. You need to design a more efficient representation of the system state that allows you to perform each move in constant time, regardless of the number of elements in both queues.

Note also that even though in the examples below `moves` is always a Python list, for the later inputs the automated tester will make `moves` a lazy **generator** object that allows you to iterate through the `moves` with a for-loop, but does not allow random access to jump back and forth or rewind the moves. This should not really affect any reasonable implementation of this function, though. (Any generator object turns into a `list` with a single function call anyway, if needed.)

`permutation`	`moves`	Expected result
`[0, 1]`	`[0, 1, 1, 0]`	`([0, 1], [])`
`[3, 1, 0, 2]`	`[0, 0, 0, 1, 3, 0, 1, 2]`	`([1, 0], [2, 3])`
`[4, 3, 0, 1, 2]`	`[4, 2, 0, 1, 3, 3, 4, 3]`	`([4], [3, 1, 0, 2])`

79. Distance from land

```
def distance_from_land(map):
```

Each square in a two-dimensional rectangular `map` grid is either `'L'` for land or `'S'` for sea. This function should return a two-dimensional list of lists of the same size as the `map`, where each element gives the integer distance from that square to its nearest land square. Compute distances between two squares as Manhattan distances: the sum of their absolute horizontal and vertical distances.

This function should be efficient even for large `map` instances to pass the automated fuzz tester. The best solution is probably to run a breadth-first search starting from all land squares simultaneously. Each `map` given to your function is guaranteed to contain at least one square of land, making all resulting distances well-defined.

`map`	Expected result

`['LSS',`
` 'SSS',`
` 'SSS']`
	`[[0, 1, 2],`
	` [1, 2, 3],`
	` [2, 3, 4]]`

`['SSSLS',`
` 'SSSSS',`
` 'SSSSS']`
	`[[3, 2, 1, 0, 1],`
	` [4, 3, 2, 1, 2],`
	` [5, 4, 3, 2, 3]]`

`['SSSSLSLLS',`
` 'LSLLLSSSS',`
` 'SSLLLLSSL',`
` 'SSLSSLSSS',`
` 'LSSLSLSSS',`
` 'SLLLSLSSS',`
` 'SSSSSLLSS',`
` 'SSSSLLSSL']`
	`[[1, 2, 1, 1, 0, 1, 0, 0, 1],`
	` [0, 1, 0, 0, 0, 1, 1, 1, 1],`
	` [1, 1, 0, 0, 0, 0, 1, 1, 0],`
	` [1, 1, 0, 1, 1, 0, 1, 2, 1],`
	` [0, 1, 1, 0, 1, 0, 1, 2, 2],`
	` [1, 0, 0, 0, 1, 0, 1, 2, 2],`
	` [2, 1, 1, 1, 1, 0, 0, 1, 1],`
	` [3, 2, 2, 1, 0, 0, 1, 1, 0]]`

This problem is adapted from LeetCode problem 1162, “As Far From Land as Possible”.

80. Smallie Bigs

```
def partition_point(items):
```

The famous quicksort algorithm, to, well, *quickly sort* the elements of the given list, starts by **partitioning** the elements of the list into two lists of "small" and "big" elements so that every element in the list of small elements is less than or equal to every element in the list of big elements. Various clever ways to perform this partition, balancing the "divide and conquer" recursion of the quicksort algorithm, have been invented.

However, in this problem, we want to determine whether a good partition point already exists for the given list of `items`. This function should determine whether the list of `items` can already be split at some position into two sublists so that every element in the left sublist is less than or equal to every element in the right sublist. This function should return the number of elements in the left sublist of the resulting partition. If multiple solutions exist, you should return the one that makes the number of elements in the left sublist as close to half of the length of the original `items` list.

The automated fuzz tester will give this function only lists where such a partition exists so that both sides end up having at least one element. Furthermore, the tester will give your function 10,000 test cases that contain up to thousands of elements, so your function needs to operate in linear time with respect to the length of the list, instead of sawing back and forth in an inner loop for each element like some kind of Shlemiel.

items	Expected result
`[0, 9, 15, 13, 14]`	2
`[5, 11, 3, 13, 18, 22]`	3
`[53, 23, 31, 23, 3, 47, 42, 22, 79, 65]`	8
`[108, 148, 107, 6, 44, 39, 130, 281, 185, 274, 175, 240, 168, 188]`	7

81. Gauss circle

```
def gauss_circle(r):
```

The Gauss circle problem of **combinatorial geometry** asks how many points (*x*, *y*) where both coordinates *x* and *y* are integers lie within the disk of radius *r* centred at the origin. (Points at the edge of the disk are counted as being within the disk.) This famous problem remains open, as no one has found a closed-form solution in terms of the radius *r*. Of course, in this course, we can still use this classic problem as an exercise in writing loops to solve the problem efficiently, at least for reasonably small *r*.

One simple way to count the points is to add them up row by row. Initialize the total count to 0 and start at the disk's topmost point (0, *r*). Whenever you are standing at the point (*x*, *y*), add the $2x + 1$ points on the row *y* to the total count, and do the same for the points in the row $-y$ whenever $y > 0$. Decrement *y* and increment *x* as long as the point (*x*, *y*) remains inside the disk. Once you have added up the points in all rows in this manner, return the final total.

r	Expected result
1	5
2	13
7	149
58	10557
1809	10280737
1000000	3141592649625

The closely related Euclid's orchard problem is otherwise the same. Still, it counts only the points (*x*, *y*) directly visible from the origin, with no other points between the origin and that point, which happens precisely when *x* and *y* are **relatively prime** and share no factors other than 1.

82. Zeckendorf decoding

```
def zeckendorf_decode(fits):
```

Fibonacci numbers often teach recursion in computer science. However, they can also serve as building blocks for all positive integers because Zeckendorf's theorem says any positive integer can be expressed as a sum of distinct, non-consecutive Fibonacci numbers in exactly one way. You can compute that representation easily with a greedy algorithm that always uses the largest Fibonacci number that still fits in the remaining number. For example, the unique representation of 72 as such a sum is 1 + 3 + 13 + 55.

As an alternative to expressing a positive integer in binary "bits" as a sum of distinct powers of two, **Zeckendorf encoding** represents this sum of distinct Fibonacci numbers as binary "fits," where ones indicate the Fibonacci numbers included and zeros indicate those excluded. Unlike ordinary positional number systems, where digits are written in order of decreasing powers of the base, Zeckendorf encoding writes the fits in increasing order of Fibonacci numbers. For example, the Zeckendorf encoding of 72 is `101001001`. This order helps greatly when encoding an arbitrarily long list of integers, each of an arbitrarily large magnitude, where the Zeckendorf encoding shines since the impossible fit pattern 11 can be used to denote the "comma" that separates two consecutive numbers in the list, the first of these two fits serving a double duty as the highest-order fit of the integer being encoded! For example, the list `[72, 6, 81, 2, 5]` would be encoded as a string of fits `101001001`1`1001`1`000100101`1`01`1`00011.

This function should decode the given sequence of `fits` back to the original list of integers. Your function can assume that the last two `fits` are always `11`.

fits	Expected result
'01001110001101011'	[10, 9, 7]
'011'	[2]
'10010000011100010010111001001101010111 1'	[95, 132, 27, 20, 1]
'00101011'	[32]
'00100101010001101000000001011011'	[749, 523, 2]

When integers are expressed in binary positional notation, encoding the separator between integers of arbitrary magnitude can get tricky. Elias gamma coding is one clever way to achieve even tighter compression than Zeckendorf encoding.

83. Sultan's daughter

```
def sultans_daughter(expression):
```

Similar in spirit and style to the “McCulloch's Second Machine” problem in the original collection of *109 Python Problems*, **Sultan's daughter** is an esoteric programming language based on the string-rewriting paradigm, devised by Raymond Smullyan himself.

Expressions in Sultan’s daughter are strings consisting of digit characters `0` to `9`, with no other characters allowed. To evaluate an expression of the form `dX`, where `d` is some leading digit character, and `X` is some string of digit characters (possibly empty), the first digit `d` determines which operation is applied to the recursively computed value of the rest of the expression `X`.

Evaluation rules for possible values of `d` are listed on the Wiki page for Sultan’s daughter at the website `esolangs.org` for the reader to consult. For example, since `19872` evaluates to `987`, `319872` evaluates to `987987`, and therefore `4319872` evaluates to `789789`. This function should iterate the given `expression` of digit characters `0` to `9` until the evaluation either terminates when none of the rewriting rules apply, or is nonterminating due to reaching some expression that has already been encountered during that evaluation. This function should return the number of iteration steps required to reach termination. In case of a non-terminating `expression`, return `-1` instead.

`expression`	Expected result
`'663653'`	`1`
`'1171112222'`	`4`
`'13115616633131152222222'`	`8`
`'11313116113114722222222'`	`10`
`'47431474312'`	`-1`

That last expression `47431474312` is actually a quine in Sultan's daughter, evaluating to itself.

84. Limited swap sorting

```
def limited_swaps(perm, swaps):
```

All **comparison sort** algorithms operate by swapping two elements according to some discipline that guarantees the resulting list will be sorted after the comparisons and conditional swaps. This problem considers an interesting restriction of sorting: the function is given a list of allowed `swaps` as tuples `(i, j)`, where `i` and `j` are position indices.

Given the list `perm` that is guaranteed to be some **permutation** of integers 0, …, n – 1, this function should determine how close to sorted order that permutation can be brought when limited to using only the given `swaps`. This function should return the **lexicographically smallest** list that is reachable from the given `perm` using only the allowed `swaps`. Note that the same individual swap may be used multiple times during the sorting process to marshal the elements towards their final resting positions.

perm	swaps	Expected result
`[2, 0, 1]`	`[(0, 1], (0, 2)]`	`[0, 1, 2]`
`[2, 3, 1, 0]`	`[(1, 2), (2, 3)]`	`[2, 0, 1, 3]`
`[0, 3, 2, 4, 1]`	`[(0, 1), (0, 2), (1, 2)]`	`[0, 2, 3, 4, 1]`
`[3, 4, 0, 6, 1, 2, 5]`	`[(1, 4), (3, 4), (5, 6)]`	`[3, 1, 0, 4, 6, 2, 5]`

This problem is “Mosey's Birthday” in the DMOJ online problem collection. (Why that problem is called that over there, well, your guess is basically as good as mine.)

85. Average run length test of randomness

```
def average_run_length(sequence):
```

As in the earlier "Poker test" problem, statistical tests to evaluate the quality of **pseudorandom number generators** first produce a long sequence of pseudorandom numbers from that generator, and then calculate various statistical quantities for that pseudorandom sequence and compare those quantities to the quantities that a truly random sequence would be expected to have.

Another such test computes the average length of the non-ascending and non-descending runs of the given sequence. For example, the sequence `[2, 2, 0, 1, 2, 6]` starts with a non-ascending run `[2, 2, 0]` of length 3, followed by a non-descending run `[0, 1, 2, 6]` of length 4. Note how the turning point element, where the direction of the run turns, is included in both runs. The average run length of this example sequence is therefore (3 + 4) / 2 = 7/2.

This function should compute the average run length of the given `sequence` of integers, and return the answer as an exact `Fraction` object. Be careful to ensure that your function also correctly handles the sequences that start with a run of equal values, forcing you to wait and see whether that first run is non-ascending or non-descending.

`sequence`	Expected result (as `Fraction`)
`[3, 1]`	`2`
`[1, 1, 1, 3, 4, 2, 5]`	`3`
`[1, 1, 1, 5, 5, 2]`	`7/2`
`[1, 3, 8, 8, 11, 9, 10, 11]`	`10/3`
`[4, 4, 8, 14, 15, 11, 11, 7, 0, 3, 1, 1, 9]`	`17/5`
`[42, 42, 42, 42]`	`4`

Students interested in this kind of stuff can also consider the expected average run length for a truly random sequence of integers from 1,..., *n*.

86. Count increasing paths

```
def count_increasing_paths(matrix):
```

You are given an *n*–by–*n* element square `matrix` that contains each one of the integers 0, ..., $n^2 - 1$ in some order. The two-dimensional square `matrix` is given as the list of its individual rows, which allows you to index the element in row `i` and column `j` with the expression `matrix[i][j]`, just like you did in your linear algebra courses.

Your task is to count how many separate increasing paths exist inside the entire `matrix` when you are allowed to start the path from any cell and keep moving to any of the four axis-aligned compass directions, as long as you always take each step into an adjacent cell that contains a higher value than the cell that you are moving away from. The `matrix` edges do not wrap around toroidally; they are solid walls extending to infinity, with nothing existing outside the matrix.

The increasing paths in the matrix have a self-similar structure that lets you solve this problem recursively. The number of increasing paths of length *k* that terminate at cell (*i*, *j*) can be computed recursively by summing the number of paths of length k – 1 that lead to neighbouring cells with a lower value than (*i*, *j*). The base case of this recursion is the count of singleton paths starting and ending at any one cell being equal to 1. You can either write a recursive function to solve this and use the `@lru_cache` decorator to **memoize** it, or you can use **dynamic programming** to fill in the table of path counts for length *k* from the previous table that contains the path counts for length *k* – 1. If you choose to take the latter route, note that you only need to maintain the **sliding window** of two such tables, assuming that you add up path counts in ascending order, since each count of paths of length *k* depends only on the counts of paths of length *k* – 1.

`matrix`	Expected result
`[[1, 0], [2, 3]]`	`11`
`[[3, 0], [1, 2]]`	`8`
`[[1, 2, 8], [6, 3, 4], [0, 5, 7]]`	`39`
`[[10, 15, 1, 12], [14, 9, 7, 0], [11, 6, 2, 13], [8, 5, 3, 4]]`	`89`
`[[36, 31, 39, 5, 17, 1, 23], [13, 35, 48, 4, 38, 18, 43], [21, 28, 10, 12, 25, 34, 6], [16, 2, 8, 19, 3, 11, 44], [41, 32, 47, 7, 37, 29, 0], [46, 14, 22, 27, 45, 26, 15], [24, 33, 40, 9, 42, 30, 20]]`	`258`

87. Cocke–Younger–Kasami parsing

```
def cyk_parser(text, rules):
```

A context-free grammar consists of **nonterminal symbols** (here, uppercase letters), of which 'A' is conventionally the start symbol, and terminal symbols (here, lowercase letters) that make up the text string. The grammar is defined by rewrite rules of the form $S \to \alpha$ that specify the sequences of terminal and nonterminal characters that the left-hand side S can be replaced with. The `rules` are given as a Python dictionary whose keys are the left-hand side nonterminals and the values are the lists of possible right-hand sides for those nonterminals. The grammar **produces** the strings of terminal characters that can be produced by applying rewrite rules in some order from the start symbol, each rule applied one at a time to one occurrence of its left-hand side in the current string of symbols.

The context-free grammar is in Chomsky normal form if the right-hand side of each rewrite rule is either a single terminal symbol or exactly two nonterminal symbols. Whether the given terminal string α can be produced from a grammar whose `rules` are in Chomsky normal form can always be efficiently determined with the Cocke–Younger–Kasami algorithm. This algorithm uses simple recursion on the current nonterminal symbol S and terminal string α, starting with the start symbol and the entire `text` in the top-level call.

The base case of recursion is when α is one terminal symbol, easily answered by checking if there is a rewrite rule $S \to \alpha$. Otherwise, check whether α can be split into two nonempty pieces $\alpha = \beta\gamma$, so that the grammar has some rewrite rule $S \to BC$ for which B produces the left piece β, and C produces the right piece γ. You can write this parsing algorithm as straightforward recursion using the `@lru_cache` decorator to tame the exponentially overlapping subproblems, or as a bottom-up **dynamic programming** approach, systematically tabulating subproblem results by translating the algorithm pseudocode from the linked Wikipedia page to Python.

text	rules	Expected result
`'ccb'`	`{'A': ['a', 'b', 'BA'], 'B': ['c', 'BC', 'b', 'CA'], 'C': ['CA', 'a', 'c', 'BC']}`	`True`
`'bccb'`	`{'A': ['BC', 'DB', 'b'], 'B': ['d', 'BA', 'c', 'b'], 'C': ['AC', 'DC', 'AB', 'a'], 'D': ['DB']}`	`False`

For some reason, whenever you talk about grammar, automata theory, and parsing like you're some fifties mathematician with a crew cut and horn-rimmed glasses, it's compulsory to use Greek letters to denote strings.

88. Avoidant subset of points on a circular line

```
def farthest_points(points, p, k):
```

Several `points` have been placed along a line segment of length `p` that has been wrapped around **cyclically** so that its endpoints are connected, making that line segment a weird little thing a bit like a circle, but not quite so. The `points` are given in sorted order in the list, and each point lies in the range [0, *p*) on the line.

Your task is to choose a subset of exactly `k` of these `points` to maximize the minimum pairwise distance among your chosen subset of points. The distances of the other pairs of points in your chosen subset do not matter, but the two points that lie closest to each other determine the solution cost. Since this distance-maximizing subset of points is not necessarily unique, your function should return the maximum distance of the two closest points in your chosen subset.

points	p	k	Expected result
[3, 5, 8, 11, 13, 20]	21	3	6
[3, 5, 7, 12, 13, 20, 21]	22	4	4
[0, 1, 3, 6, 11, 15, 16, 22]	24	4	5
[0, 1, 7, 10, 11, 13, 14, 19, 20, 27]	29	4	6
[3, 4, 9, 10, 12, 14, 18, 20, 21, 23, 24, 25, 28, 31, 34, 40, 42]	43	4	10

89. Ehrenfeucht–Mycielski sequence

```
def ehrenfeucht_mycielski(prefix, k):
```

The blog post “Guess again? The Ehrenfeucht–Mycielski sequence” by Jim Propp offers a lively discussion of another famous self-referential sequence of bits in which iterating a simple rule generates highly chaotic and unpredictable behaviour. This function should generate and return the sequence extended by *k* bits from the given initial `prefix`.

The bits of the prefix are extended individually according to the following rule. Start by finding the **longest proper suffix** of that prefix that occurs somewhere else inside that same prefix. Overlapping occurrences are accepted, so the longest suffix of the sequence `'010101'` that occurs somewhere else would be `'0101'`, not `'01'`. In the edge case where the current prefix is either `'0'`, `'1'`, `'01'` or `'10'`, use the trivial empty suffix that occurs at the beginning of the prefix.

Once you have found this longest proper suffix that works, find the rightmost position where this suffix occurs, excluding the occurrence at the very end. For the empty suffix, when no suffix characters are matched, use the position at the beginning. Extend the prefix with the negation of the bit that follows this rightmost earlier occurrence. For example, in the previous example prefix `'010101'`, the bit following the rightmost earlier occurrence of its longest suffix `'0101'` is `0`, making the new extended prefix `'0101011'`.

The test cases will be nontrivial but short enough so that a good optimized search will pass the tests within the time limit. There is no need to implement any kind of suffix tree or other such structure.

prefix	k	Expected result
`'0'`	`10`	`'01001101011'`
`'01'`	`14`	`'0100110101110001'`
`'101'`	`21`	`'101100101000111011110000'`
`'01000'`	`42`	`'010001101011100100111101100001010000001110100101'`

The interested reader can compare this sequence to **Kimberling's repetition-resistant sequence** included in the Additional Python Problems collection, and also to the Linus sequence included in this author's collection of Java coding problems for the course CCPS 209. Both sequences produce self-avoidant chaos using similar but subtly different approaches.

90. Diamond sequence

```
def diamond_sequence(k):
```

One example in the book “Mathematical Diamonds” by the late great Ross Honsberger features a beautiful, infinitely intricate sequence defined by a simple self-referential rule. As a mathematician rather than a computer scientist, the sequence positions start at 1, with the first element equal to 1. After that, the element in each position *k* equals the smallest previously unseen positive integer that makes the sum of all elements up to and including that position divisible by *k*. The reader can verify that the sequence starts as

1, 3, 2, 6, 8, 4, 11, 5, 14, 16, 7, 19, 21, 9, 24, 10, 27, 29, 12, 32, 13, 35, 37, 15, 40, ...

This function should return the element in position k of this sequence. For simplicity, the automated tester will call this function for values of *k* from 2 to 999999 in this order, so your function does not need to remember the entire sequence so far, but can store the previous element and the set of values seen so far in `global` variables that persist between the function calls.

Furthermore, since this set of values seen so far consists of positive integers and is **monotonic** (that is, an element once seen will never be unseen), you should also maintain in a global variable the lowest unseen positive integer that has not appeared in the sequence so far. You can save plenty of memory and some running time by removing the elements from the set of seen elements that are lower than this lowest unseen element. (Modify the set inclusion tests accordingly in the code.)

k	Expected result
2	3
7	11
11	7
1000	1618
1618	1000

This sequence has an amazing property, as the table above illustrates. If the element in position *k* equals *m*, then the element in position *m* equals *k*. In other words, this sequence is a **permutation** of all positive integers that, after position 1 maps to itself, consists entirely of two-cycles. This fact is all the more astonishing in that generating this sequence seems to have nothing to do with this property. Yet it still chugs out the elements in a seemingly random order of these neat two-cycles as surely as a train on its tracks, never slipping or missing a beat on its journey to infinity.

91. Minimize largest triangle

```
def min_max_triangle(points):
```

You are given a list of 3*n* points on a two-dimensional plane. Your task is to divide these points into *n* groups of three points each so that when each group of points is considered a triangle, the area of the largest triangle is as small as possible. This function should return the area of the largest triangle multiplied by two, to ensure that the result is always an integer. For example, if the area of the largest triangle is 5 1/2, your function should return 11.

This problem can be solved recursively by realizing that the first point must be part of some triangle, so loop through the possibilities for choosing the other two points for that triangle. For each such triplet, compute the best grouping for the remaining points. Once the area of some triangle created in the recursion is not smaller than that of the best solution found so far, you can terminate that branch of the search immediately and return to the previous level.

The example program `geometry.py` in the `ikokkari/PythonExamples` repository contains a utility function that computes the **cross product** of three points that gives twice the signed area of that triangle. Note that depending on the orientation of the three points, the signed area can be negative, so remember to take the absolute value of this quantity to use as the area of the triangle.

```
def cross(p1, p2, p3):
    (x1, y1), (x2, y2), (x3, y3) = p1, p2, p3
    return (x2 - x1) * (y3 - y1) - (y2 - y1) * (x3 - x1)
```

points	Expected result
`[(2, 4), (1, 2), (4, 3), (0, 3), (3, 0), (2, 2)]`	4
`[(10, 15), (13, 14), (11, 0), (14, 1), (4, 12), (12, 9), (0, 12), (10, 10), (15, 3), (1, 3), (3, 5), (0, 11)]`	6
`[(6, 12), (25, 16), (12, 17), (5, 14), (20, 17), (23, 19), (11, 22), (8, 9), (3, 3), (8, 22), (22, 0), (16, 18), (21, 18), (4, 17), (5, 2)]`	13

92. Cutting sticks

```
def cutting_sticks(sticks):
```

A single stick can break easily, but together in a bunch, they can do mighty things. Consider a bunch of integer-length `sticks` whose sum of lengths equals $n(n + 1) / 2$ for some positive integer n. Also assume each stick has length at least n. Under these assumptions, you can always cut the `sticks` into n pieces with lengths 1, …, n, with no glue needed.

The cuts to achieve this are not necessarily unique. For example, if given the sticks `[5, 10]`, we can keep the stick 5 as it is, and split the stick 10 into 1 + 2 + 3 + 4. However, we could also split stick 5 into 1 + 4 and split stick 10 into 2 + 3 + 5. The number of cuts is always the same in every solution (DUCY?). Still, we want to find the best way to cut the sticks by distributing these cuts evenly across the sticks, with a cost function where cutting a stick into k pieces costs $(k - 1)^2$ dollars. (For example, cutting a stick into three pieces would cost $(3 - 1)^2 = 4$ dollars.) Under this cost function, the second way to cut the sticks is better, with a cost of 1 + 4 = 5, compared to the first, more unbalanced solution's cost of 0 + 9 = 9.

This function should return the cost of the optimal way to cut the `sticks` to produce the required n pieces. You can solve this with **backtracking** that examines the possible ways to extract a piece of length k from the given sticks, and for each such cut, recursively explores ways to extract pieces of length $k - 1$ from the remaining sticks. At the top-level call, $k = n$, and the base case is $k = 0$, with all required pieces cut. As usual, you are free to tune this backtracking algorithm however you like; the only rule is that your code must be standard Python and produce the correct expected results for every automated test case.

pieces	Expected result
[5, 10]	5
[12, 14, 10]	9
[18, 11, 15, 11]	10
[58, 72, 19, 22]	50

Dropping the assumption that each stick is at least length n gives up the guarantee that a legal solution exists. In fact, determining whether a legal solution exists for a given set of sticks is an NP-complete decision problem.

93. Optimal text filling

```
def optimal_ab_filling(text, ab, ba):
```

You are given a `text` string guaranteed to consist only of characters `'a'` and `'b'`, and the period character that acts as a placeholder for an empty location. Your task is to fill in these empty locations with characters `'a'` and `'b'` of your choice to minimize the cost over the entire string, and return the minimal cost that you achieved. Two adjacent characters that are equal incur no cost, whereas `'ab'` incurs the cost `ab`, and `'ba'` incurs the cost `ba`.

This problem can be solved recursively with a function that takes two parameters: the position `i` in the `text` string and the character placed at the previous position `i-1`. This function should try the possible characters at position `i` (whenever the `text` already contains a non-placeholder character in that position, the choice is forced without branching) and choose the character that minimizes the total overall cost continuing from that position. Since this recursion visits the same subproblems exponentially many times, use the `lru_cache` magic to prune this into a linear number of branches.

text	ab	ba	Expected result
`'b.a'`	3	4	3
`'b.b.a'`	1	2	2
`'....ab'`	4	1	4
`'b.aaa..a'`	3	2	2
`'..a.baa.'`	5	5	10

This problem is adapted from the Google Code Jam 2021 problem “Moons and Umbrellas”.

94. S-expression evaluator

`def s_eval(expression):`

Lisp, the great-granddaddy of functional programming languages, is known for its homoiconic syntax, which treats code and data the same way: fully parenthesized **S-expressions** written in **prefix notation**. An s-expression is either an **atom** (such as 42) or a structure `(f a1 ... an)` where `f` is a function symbol, and each `ai` is an s-expression whose value becomes that argument to the function `f`. S-expressions are evaluated from the **inside out**; for example, (* (+ 1 1) (- 7 3)) first simplifies to (* 2 4), then to `8`.

This problem asks you to write a function that evaluates the given s-`expression`, limited to the case where the only functions are the three basic arithmetic operators '+', '-', and '*' that always take exactly two arguments each, and the atoms are nonnegative integers. Your function does not need to be able to deal with syntactically incorrect or invalid S-expressions.

In a more advanced course, you could solve this with a classic recursive descent parser, but you can also solve it more simply with one loop and an initially empty `list` used as a **stack**. Loop through the characters of the `expression` and skip all whitespace and left parenthesis characters, but `append` each arithmetic operator and integer atom that you encounter to the stack. Whenever you encounter a closing parenthesis, `pop` the preceding two arguments and the operator from the stack, perform that corresponding arithmetic operation on those two arguments, and `append` the result back to the stack. Once you reach the end of a legal S-expression, the stack will contain exactly one integer atom to return as the result. Also make sure to correctly handle the special case where the `expression` is a single integer atom.

expression	Expected result
42	42
(+ 2 2)	4
(* (* 0 4) (+ 2 3))	0
(* (* 58 (- 47 49)) (* 40 (- 40 49)))	41760

95. Expand recursively run-length encoded string

```
def expand_string(text):
```

Run-length encoding can compress text strings with repeated patterns by storing each repeated pattern once with its repetition count. If the pattern contains repeated subpatterns, those subpatterns can also be run-length encoded, and this recursive structure can be nested arbitrarily deep.

Let us use the syntax `k[pattern]` to indicate that the `pattern` is to be repeated *k* times. For example, `2[abc]` would expand to `abcabc`. If the pattern itself contains such recursive repetitions, they are expanded inside out before the outer expansion is performed. For example, `2[a3[bc]]` would first expand to `2[abcbcbc]`, which then expands to `abcbcbcabcbcbc`.

This function should expand the given `text` in this manner. Your function may assume that the `text` will follow the syntax rules of this encoding, so it does not need to be able to deal with any malformed input strings. As in the previous problem of evaluating recursively nested S-expressions, you could, of course, use recursion, but you don't need a full-on recursive descent parser, since a stack-based approach that runs in a single linear for-loop through the text without backtracking can handle this job. Ain't properly parenthesized expressions just dandy?

`text`	Expected result
`'4[x]'`	`'xxxx'`
`'4[2[u]]xqk4[r]2[i]'`	`'uuuuuuuuxqkrrrrii'`
`'t5[4[n]]'`	`'tnnnnnnnnnnnnnnnnnnn'`
`'l3[io]bs2[d]iiyg2[2[kw]i]'`	`'lioioiobsddiiygkwkwikwkwi'`

This cute little problem is the LeetCode problem 394, “Decode String”.

96. Longest common strictly increasing subsequence

```
def lcsis(items1, items2):
```

The longest common subsequence problem is a well-known first example used to teach dynamic programming. (In fact, you can recognize the algorithm courses and instructors whose learning was based on the White Book from the fact that they use this particular problem as the first example of dynamic programming!) This problem asks you to find the longest possible sequence of items that is simultaneously a **subsequence** of the two lists `items1` and `items2`. For example, the longest common subsequence of the sequences `[3, 9, 1, 0, 17, 2]` and `[9, 0, 5, 6, 2]` would be `[9, 0, 2]`.

For two sequences of length *n* and *m*, this problem can be solved with a recursive function L(n, m) with base cases L(*n*, 0) = L(0, *m*) = 0. For the general recursive case L(*n*, *m*), where *n* and *m* are the lengths of the two respective strings, the result is the maximum of the three numbers L(*n* – 1, *m*), L(*n*, *m* – 1) and 1 + L(*n* – 1, *m* – 1), where the third case applies only if the last elements of both sequences are equal. As usual, you can make this recursion efficient through memoization or dynamic programming by tabulating the results bottom up.

However, this problem adds an extra twist to this classic recursion example: your recursive function must find the longest common subsequence whose elements are strictly increasing. Your recursive function needs three parameters, with the third parameter being the upper limit for the next element in the subsequence. Since this subsequence is not necessarily unique, return only its length.

items1	items2	Expected result
`[7, 13, 1]`	`[2, 15, 3, 8, 2]`	`0`
`[8, 17, 13, 17]`	`[2, 23, 18, 5, 8, 13, 2, 17]`	`3`
`[47, 39, 78, 4, 35, 63, 6, 81, 3, 90, 112]`	`[4, 124, 102, 35, 81, 90, 111, 112]`	`5`

97. Maximum palindromic integer

```
def maximum_palindrome(digits):
```

Here is a cute little puzzle from a technical interview problem collection for which writing the actual code is not too difficult, but the difficulty is in coming up with the algorithm to solve the problem under the time limit of solving the question under pressure of getting the job and securing your future on the good side of the barbed wire fence in the diminishing race to the bottom that the entire computing industry has suddenly turned into since the golden years of the youth of this author. Yes, children, gather around to hear how there once was a peaceful time when you could get an $80K entry-level job (purchasing power equivalent to about $200K total compensation in 2025 money) for merely knowing how to write loops in Visual Basic.

Given a string of `digits` you can use, construct the largest possible integer that is a palindrome (reads the same left to right as right to left). The resulting number may not contain any leading zero digits, but may contain zeros inside the number itself. Each digit can be used as many times as it appears inside the `digits` string.

digits	Expected result
'0'	0
'011'	101
'8698'	898
'123123123'	3213123
'225588770099'	987520025789

98. Optimal palindromic split

```
def palindrome_split(text):
```

Any `text` string can be split into disjoint contiguous pieces so each piece is a palindrome, that is, it reads the same left to right and right to left. Of course, making each piece a single character trivially satisfies the definition of a palindrome, but some strings allow a more elegant partition with significantly longer pieces.

Following the general principle that you get what you reward, let us define each palindromic piece of length k as worth k^2 points. For example, a piece with five characters would give you 25 points, much better than using those same five characters as five separate pieces worth a measly one point each. This function should determine the optimal way to split the given `text` string into palindrome pieces to maximize the overall score, and return that maximal score.

This problem cannot be solved with a **greedy** approach where you make the first palindromic piece as long as possible, and then try to maximize the score over the remaining `text` in the same greedy vein. The greedy choice for the first palindrome might consume characters needed to make the next piece as long as possible, as in `'SOSAIPPUAKAUPPIAS'`. You should instead use **dynamic programming** to compute the maximum score M(i) as the maximum over the immediate scores of each possible palindrome that starts from position i, plus the maximum score achievable from the position after that initial palindrome. You can solve this as a recursive function using the `@lru_cache` decorator to **memoize** it to avoid iterating through the same subproblems an exponential number of times, or you can go hardcore and tabulate the maximum scores backwards from the end of the `text` to the beginning, and then return `table[0]` as your final answer.

`text`	Expected result
`'AXA'`	`8`
`'ABAB'`	`10`
`'())('`	`16`
`'CABCCBA'`	`37`
`'CACAACCAABBAACCAA'`	`205`
`'AACBAACBCAACCA'`	`60`

Also, don't forget that your algorithm isn't actually dependent on the acceptability criterion being a palindrome or the scoring function being the square of the piece length. Still, the dynamic programming algorithm is the same for any acceptability criterion and scoring function you plug in.

99. Semiconnected guys

```
def is_semiconnected(edges):
```

A **directed graph** consists of nodes numbered 0, ..., *n* – 1. The `edges` are given as a list where edges[u] is the list of edges emanating from node *u* to its immediate neighbours. This function should determine whether the graph is semiconnected; that is, for each pair of nodes u and v, there exists a directed path from u to v or from *v* to *u*. Note that this question is not the same as testing whether the graph is fully connected when each edge is treated as undirected, since the edges in each directed path must still point consistently in the same direction. Each node is considered to be automatically connected to itself with an empty path.

The simplest way to solve this problem would be to implement the classic Floyd–Warshall all-pairs reachability algorithm to determine the reachability between all pairs of nodes simultaneously, and then verify from the resulting reachability matrix that the nodes of each pair in the graph are reachable from each other in at least one direction. Because the paths connecting nodes inevitably share many common subpaths, the Floyd–Warshall algorithm is superior to running a breadth-first search from every node and then collating the results.

edges	Expected result
`[[3, 2], [2], [1, 3], []]`	`True`
`[[], [0], [4, 0], [4, 1], [1]]`	`False`
`[[3, 2], [4], [], [1], [0]]`	`True`
`[[4], [5], [1, 0, 5], [2], [0, 5], [4, 1, 3], [5, 3]]`	`True`

100. Falling squares

```
def falling_squares(squares):
```

Starting with a flat surface at height zero, `squares` fall straight down from a large height, one at a time, like Tetris. Each square is a two-tuple `(x, w)`, where `x` is the *x*-coordinate of the square's left edge, and `w` is its width (and thus also its height). Each square falls straight down until some non-zero part of it lands on a flat surface, either the floor or some previous square. The surfaces between squares are infinitely sticky, so the square sticks where it lands, again in Tetris style, without tipping over, regardless of any dramatic overhang of the column squares set up so far.

Given the list of `squares` in the order in which they fall one by one, this function should return a list of the same length whose each element equals the largest height that the entire resulting structure reaches after dropping that square. Your function should somehow keep the surface's current shape, and efficiently update that surface after each square.

This problem is a hardened version of problem 699 of LeetCode, “Falling Squares”. Their version keeps the coordinates and square sizes small enough for you to simply maintain a **height field**, a list of current height values for the range of x-coordinates that the squares fall on, and update it with a trivial for-loop for each new square. In our version, after a smooth start with test cases small enough to help your debugging, the automated tester will start shooting these numbers up exponentially. Instead of using a height field, you therefore need to set up some kind of higher-level representation for the outline of the shape created by the squares so far that remains efficient in both time and space, even for square positions and widths up in the gazillions.

`squares`	Expected result
`[(4, 3), (6, 1), (5, 1)]`	`[3, 4, 4]`
`[(8, 1), (2, 1), (11, 1), (3, 2)]`	`[1, 1, 1, 2]`
`[(4, 3), (6, 3), (2, 1), (2, 2), (13, 1)]`	`[3, 6, 6, 6, 6]`
`[(7, 2), (3, 2), (7, 1), (9, 3), (5, 2), (9, 3), (7, 4), (10, 1)]`	`[2, 2, 3, 3, 3, 6, 10, 11]`

101. Tchuka Ruma

```
def tchuka_ruma(board):
```

Tchuka Ruma is an interesting solitaire variation of Mancala, the family of **sowing games**. The `board` consists of a **store** in position 0 and **holes** in positions 1, ..., *n* – 1. The store starts empty, and each hole starts with some number of pebbles. Your goal is to get as many pebbles as you can in the store before you inevitably paint yourself into a corner with no more possible moves. This function should return the maximal number of pebbles that end up in the store.

A move consists of picking up all pebbles from a nonempty, non-store hole and sowing them into holes starting at the position immediately preceding the chosen hole and continuing leftward one pebble at a time. If this sowing process extends past the left edge of the board after placing a pebble in the store, it continues cyclically in the holes at the right end of the `board`. This sowing move can then end in three possible ways:

- If the last pebble ends up in the store, that move is completed. The player can choose his next move freely from the available non-empty holes.
- If the last pebble ends up in an empty non-store hole, the game is over with the final score equal to the number of pebbles that ended up in the store.
- Otherwise, the move is not over, but continues by the player picking up all the pebbles in that last hole and sowing them leftward in the same fashion, with the same end conditions.

board	Expected result
`[0, 3, 0]`	`1`
`[0, 3, 1]`	`3`
`[0, 3, 2, 4]`	`7`
`[0, 1, 4, 4, 5]`	`13`
`[0, 5, 4, 7, 7, 0, 0, 8, 1]`	`2`
`[0, 5, 9, 3, 0, 1, 9, 2, 3]`	`32`

Since the sowing process can only move these pebbles leftward and cannot skip over the store from which no pebbles ever get picked up, we see that even though a single move can consist of several stages of picking up and sowing pebbles, each move must eventually terminate after a finite number of steps, since during each round trip around the board, the number of pebbles available for seeding will decrease by one.

102. Slater–Vélez sequence

```
def slater_velez(k):
```

This problem closely resembles the earlier “Diamond sequence” problem. It can be solved with the same technique, even though this self-referential process chugs out a sequence of positive integers that is even more chaotically whimsical than the diamond sequence. The numbering of sequence positions again starts at 1 instead of 0, and the first element in that first position also equals 1. After that, each element is always the smallest positive integer *n* that has not appeared so far in the sequence, under the constraint that the absolute difference of *n* and its immediate predecessor in the previous position of the sequence also has not previously appeared as the absolute difference of any two consecutive elements earlier in the sequence. With some quick arithmetic, the reader can verify that this sequence starts its journey as

1, 2, 4, 7, 3, 8, 14, 5, 12, 20, 6, 16, 27, 9, 21, 34, 10, 25, …

Since the sequence of absolute differences between the consecutive elements of the previous sequence begins as

1, 2, 3, 4, 5, 6, 9, 7, 8, 14, 10, 11, 18, 12, 13, 24, 15, …

This function should return the value in position *k* of this sequence. The automated tester will again give your function consecutive values of *k* starting from 2 upwards, so your function can store the previous element and the sets of previously seen values and previously seen absolute differences as global variables. Since both sets are monotonic, you can use the same technique as in the diamond sequence problem by tracking the lowest unseen value in each set, which eliminates the need to store elements below that value explicitly and greatly reduces this function's memory use.

103. Odds and evens

```
def odds_and_evens(first, second):
```

This function takes two strings, first and second, both guaranteed to consist only of binary digits `'0'` and `'1'`. These strings are to be converted to lists of positive integers so that each `'0'` becomes an even integer and each `'1'` becomes an odd integer, under the two constraints that (a) both resulting lists must be in ascending sorted order and (b) each integer may appear at most once in these two lists. Your task is to determine, over all possible ways to convert these strings into lists of integers under these constraints, the minimum possible value of the largest integer to appear in either list.

first	second	Expected result
'1111'	'1'	9
'000111'	'000111'	12
'0001'	'0001'	9
'10111011'	'10111011'	21

This cute little problem is the problem “Zrinka” in the DMOJ online problem collection. It is a bit harder than it initially seems because the author misunderstood his own problem and made an error in the original version that one tyro student pointed out. However, when this specification was given to Claude Sonnet 4, it made the same error as the original author!

104. The prodigal sequence

```
def front_back_sort(perm):
```

When you can repeatedly swap any two elements in the list, many efficient comparison sort algorithms can sort any list of items into sorted order. However, suppose that your moves are restricted to picking any one element from the list and moving that element either to the front or to the back of the list. Given the `perm`utation of integers 0, …, *n* – 1, how many such moves would be needed to rearrange the elements of that permutation in sorted order?

Of course, *n* – 1 moves always trivially suffice by repeatedly moving each element to the back, starting from one and proceeding in sorted order. But how do you recognize if some smaller number of moves would also work?

perm	Expected result
`[0, 1]`	`0`
`[1, 0, 2]`	`1`
`[0, 2, 3, 1]`	`2`
`[2, 1, 3, 4, 0]`	`2`
`[2, 1, 4, 0, 3]`	`4`
`[0, 1, 5, 2, 4, 3]`	`2`

This problem also benefits from thinking instead of brute-forcing all possible move permutations.

105. Descending suffix game

```
def descending_suffix_game(board, n):
```

This cute little problem was apparently used as a job interview question for a quant position at Jane Street, the trading firm known for its love of puzzles (solutions). You have a shuffled deck of *n* cards marked 1, …, *n*, and are invited to play a blackjack-type game where you draw cards randomly from the deck, one at a time, to build a board where, unlike blackjack, the order of the cards matters. Each round, you can choose to stand with your current board or, if the deck still has cards, hit to draw the next random card and append it to the end of your board. Once you choose to stand, your final score equals the sum of the cards in the board's longest descending suffix. For example, if you choose to stand with the board `[9, 3, 8, 4, 2]`, your final score would be 8 + 4 + 2 = 14.

This function should determine the expected value of the given `board`, assuming the player chooses moves optimally, analogous to a blackjack player mechanistically following the basic strategy card to maximize expected return. Under the basic assumptions of sufficiently small stakes in a repeated game so that your utility curve of points is linear, the expected value and thus the expected utility of the given `board` equals max(*s*, *h*), where *s* is the value of standing with the descending suffix that you have so far (easy enough to compute). The quantity *h* is the expected value of hitting, averaged over all possible next cards, assuming you continue making optimal decisions to stand or hit regardless of what the gods of randomness give you. You should do all calculations with exact `Fraction` objects.

board	n	Expected result
`[4, 1, 2]`	`5`	`13/2`
`[1, 4, 3, 7]`	`7`	`77/6`
`[11, 1, 9, 5, 12]`	`12`	`100339/5040`
`[12, 10, 7, 5, 3]`	`12`	`37`

The original question asked to calculate the expected value of the game for *n* = 9 from the empty board. This turns out to be 2749727/181440, approximately 15.155, which agrees with the commenters in the Math Overflow thread linked above.

106. The Borgesian dictionary

```
def find_all_words(letters, words):
```

Wordscapes, a popular phone and tablet game, gives players a bunch of `letters` to be used to create as many words as possible. Since this mindless task of patience and memory, iterating through all possibilities, seems better suited to a machine than a person, it's time to automate it and relieve humans of at least one burden imposed on us by modern times. This function should return the alphabetical list of all legal `words` that can be created from these `letters`. You can use each letter in the word as many times as it appears in the `letters`. Since the word list that we use in these problems is pretty huge, this function should return only the words that have at least seven letters. For simplicity, your function may assume that both `letters` and `words` are given in sorted order.

This function should be a relatively simple backtracking exercise: build the result word one character at a time by looping over the remaining letters and recursively trying to extend the word with each letter until you reach a dead end. Make sure also not to scrub through redundant search paths of duplicated letters in the same level of recursion, and use **binary search** from the Python standard library `bisect` module to quickly determine whether the word that you have built up so far is itself legal, and whether it could in principle be extended into a longer word. The efficiency of all backtracking algorithms depends on their ability to turn back as soon as possible once they have painted themselves into a dead-end corner.

`letters`	Expected result (using the wordlist `words_sorted.txt`)
`'aghimoopss'`	`['agomphosis', 'amishgo', 'gaposis', 'gomphosis', 'goosish', 'mishaps', 'samshoo', 'shamois', 'shampoo', 'shampoos', 'sophism']`
`'aeekrstuy'`	`['aesture', 'austere', 'estuary', 'euaster', 'euskera', 'keyseat', 'keyster', 'restake', 'retakes', 'sakeret', 'sautree', 'streaky', 'turkeys', 'tuyeres']`
`'aabcgkostt'`	`['attacks', 'catboat', 'catboats', 'costata', 'gasboat', 'sackbag', 'tabasco']`

Alternatively, you can preprocess the wordlist into a global data structure that maps each word to its Gödel number (the product of prime numbers that correspond to letters in that word), to then determine with integer arithmetic whether a given word can be constructed from `letters`.

107. The Dictorian borgesitory

```
def maximum_word_select(words, letters):
```

Like an upside-down version of the previous “Borgesian dictionary” problem, this function takes a list of words and a multiset of letters you can use. Your task is to choose a subset of `words` such that you can form each chosen word from the given `letters`, using each letter at most as many times as it appears. The score for the subset of words is the sum of the lengths of those words, so longer words are better. This function should return the highest total score that can be achieved with the given `words` and `letters`.

This problem is a pretty basic backtracking recursion (you either take in each word or leave it). Still, this problem is also an opportunity for you to try out the excellent `Counter` data structure in the Python standard library `collections` module to keep track of the remaining letters and quickly determine if the current word can still be constructed from your remaining letters.

words	letters	Expected result
`['hymnody', 'lycodes', 'phrasal', 'chams']`	`'aaabccddehhjllmnoopqrsssyy'`	19
`['dicht', 'dangler', 'rosal', 'ostend']`	`'aabbddddeeghillnoorstyy'`	12
`['nonfluids', 'ululate', 'tentability', 'shook', 'ticktacker', 'spectacularism', 'bourre', 'inexposure', 'sassan']`	`'aaaaaabbccccddeeeeeeeffiiiiiikkkllllmmnnnnoooooppqrrrrrrsssssssttttttuuuuuwxxxy'`	60

108. The art of strategic sacrifice

```
def pick_it(items):
```

You are given a list of `items`, each item an integer. The list contains a total of *n* items, and you need to perform the following step exactly *n* – 2 times: choose any one of the `items` that is neither the first nor the last remaining item, and remove that item from the list. At each such removal, your score increases by the square of the current predecessor of that item plus the item itself, and decreases by the square of the current successor of that item. This function should compute the highest score achievable by performing these *n* – 2 item removals in the optimal order.

This problem is adapted from “Pick it” in the DMOJ online problem collection by adding squaring and subtracting neighbouring elements to make it more nonlinear.

Like the previous problem, “The Borgesian Dictionary”, this is a basic backtracking exercise. Each recursion level loops through the remaining elements and, for each element, recursively tries to build the rest of the score from the remaining elements. To allow the code to find the current predecessor and successor of each element in constant time, you should use the **dancing links technique** with two lists `prev` and `succ`, both of length *n*. The elements `prev[i]` and `succ[i]` give the current predecessor and successor of the element in position *i*, respectively. To remove the element at position i, update its neighbours' predecessor and successor information, then restore that information after the backtracking recursion returns.

`items`	Expected result
`[1, 4, 3, 5]`	`-25`
`[1, 5, 7, 1, 6, 2, 2]`	`142`
`[3, 7, 1, 2, 2, 5, 6, 7, 7]`	`117`
`[10, 8, 1, 6, 7, 6, 6, 7, 9, 9]`	`481`

109. Colonel Blotto and the Spymaster

```
def colonel_blotto(friend, enemy, prize):
```

The “Colonel Blotto game” is an umbrella term for a family of games in which players commit limited forces to separate battlefields before knowing how the enemy will allocate theirs. Once both sides make this commitment, each battlefield's prize goes to the player who allocated more resources there, winner-take-all.

In our version of such a game, the `friend` list gives the strengths of *n* units to be distributed over *n* battlefields, listed in descending order. However, instead of allocating these units under fog of war, our valiant Spymaster has bravely run through the eye of the needle to acquire the complete allocation of enemy forces across these battlefields! This function should compute the maximum total prize available by allocating our units wisely with this knowledge. To ensure no ties on any battlefield, our heroic forces all have even strength, while those dastardly invaders all have odd strength. You must allocate exactly one unit to each battlefield, and are not allowed to combine multiple units into one for a larger strength. All three lists, `friend`, `enemy` and `prize`, are guaranteed to have the same number of elements.

A common principle of all Colonel Blotto games (for example, electoral **gerrymandering**) is that if you are going to lose some battlefield, you really want to lose it by as much as possible, since this doesn't make the enemy's gain there any larger while the enemy needlessly wastes its power to fight air. In the same vein, if you plan to win a battlefield, try to eke out as small a victory as possible. This idea gives you a simple two-way choice for each battlefield: either give it up by assigning your weakest remaining unit there, or choose to win by assigning your weakest remaining unit that can beat the enemy force the enemy assigned there.

`friend`	`enemy`	`prize`	Expected result
`[14, 10, 6]`	`[9, 13, 21]`	`[7, 1, 5]`	`8`
`[18, 12, 4, 2]`	`[9, 3, 21, 17]`	`[7, 3, 1, 12]`	`22`
`[18, 16, 16, 14, 12, 8, 6, 4]`	`[11, 9, 13, 5, 17, 19, 17, 21]`	`[1, 3, 15, 11, 7, 2, 4, 7]`	`37`

Bonus problem 110: The way of a man with array

```
def pebblery(predecessors):
```

Jack, an outwardly respectable Victorian gentleman who is secretly a malignant narcissist, knows *n* ladies numbered 0, ..., *n* – 1. The social hierarchy of these ladies forms a **directed acyclic graph** so that each lady `i` has immediate `predecessors[i]` who are all numbered higher than her. Jack's ultimate goal is to claim lady 0 in his harem for ultimate validation.

Jack's demented courtship game proceeds as follows. Each round, Jack can make one of two moves. He can successfully add to his harem any lady whose every immediate predecessor is currently in his harem. (Thus, if some lady has no predecessors, Jack can claim her in his harem at any time.) Alternatively, he can coldly discard any lady from his harem. Thanks to Jack's well-honed skills of psychological manipulation, any lady so discarded can later be courted to rejoin the harem under the same precondition of all her immediate predecessors being in Jack's harem at that moment, making her powerless to resist Jack's incessant hoovering and love bombing with promises of how it will all be totally different this time.

However, being paranoid that the women of his harem are secretly scheming together against him, Jack wants to minimize the size of his harem at any one time on the path of acquiring the favour of the lady number 0, not including the lady zero herself in this count since Jack doesn't give a jack about anything after that final conquest and will be dumping the rest of his harem anyway. The total number of moves required to reach the ultimate lady is irrelevant to him; mastering the delicate placement of favours along the way is reward enough anyway. Given the social ranking `predecessors` structure of these ladies, compute Jack's optimal courtship strategy that minimizes the maximum number of ladies in his harem at any given time, and return that minimum. Strategic removal and replacement of ladies are key to success, but the search complexity increases dramatically when multiple paths lead to the intermediate goal. As any skilled practitioner surely knows, a direct approach is rarely optimal.

`predecessors`	Expected result
`[[1, 3], [2], [3], []]`	2
`[[1, 2], [2], [3, 4, 5], [4, 5], [], []]`	4
`[[1, 2, 4, 5], [4, 5, 6], [3, 4, 6], [], [5], [6], []]`	5
`[[1, 2, 6], [3, 5, 6, 8, 9], [], [4, 10], [8], [6, 7, 8], [9], [11], [11], [], [], []]`	6

Bonus problem 111: Levine’s sequence

```
def levine_sequence(items, k):
```

This excellent problem comes from the Numberphile video “The Levine Sequence" that the reader might want to watch before embarking on this coding problem itself. Given a starting list of `items`, each item a positive integer, you will perform the following operation `k` times. Counting from the end of the current `items` list, these individual items represent the number of consecutive occurrences of the corresponding positive integer in the new `items` list. This function should return the sum of elements in the resulting `items` list after this transformation has been iterated `k` times.

For example, starting with the list `[3]`, the first iteration yields `[1, 1, 1]`, with the element 1 occurring three times. The second iteration turns this into `[1, 2, 3,` with each of the three elements 1, 2 and 3 occurring exactly once. The third iteration will turn this into the longer list [`1, 1, 1, 2, 2, 3]`. As the reader can verify, one more iteration step will then turn this into the list `[1, 1, 1, 2, 2, 3, 3, 4, 5, 6]`.

Since the new `items` list contains exactly `sum(items)` elements after each iteration step, the list grows exponentially. The difficulty, therefore, is finding a more compact representation for the intermediate lists, most likely using some variation of **run-length encoding**. Since the items are guaranteed to be consecutive positive integers after the first iteration, this encoding can and should be made even more compact. However, despite the tantalizing simplicity of this construction, as explained in the post “Levine's sequence" in Futility Closet, nobody has yet managed to compute the last term of Levine’s sequence after a measly eighteen steps starting from the initial list `[1, 1]`.

items	k	Expected result
[3]	4	28
[2, 1]	2	9
[3, 3, 2]	4	7868
[4, 4, 1, 4, 1]	6	58361987124448

Bonus problem 112: Post correspondence problem

```
def post_correspondence_problem(first, second, lo, hi):
```

The Post Correspondence Problem is famous in string matching and was first discovered in 1946, after years of research exploring the theoretical limits of computability during the totalitarian World War. Given two lists, `first` and `second`, of equal length of strings made of letters `a` and `b`, such as `['a', 'ab', 'bba']` and `['baa', 'aa', 'bb']`, the problem is to find some nonempty sequence of position indices so that concatenating the elements of both of these lists separately, as listed in this sequence, produces the same result for both lists. For example, as the reader can quickly verify, the position sequence `(2, 1, 2, 0)` produces the same result string `'bbaabbbaa'` and `'bbaabbbaa'` for both of these lists, here colour-coded for your convenience.

This function should determine whether there exists a position sequence that produces the same result for the `first` and `second` lists whose length is at least `lo` characters and at most `hi` characters. This problem should be a pretty straightforward exercise in backtracking, but you should again look for ways to prune the search and alternatives at each level of recursion.

first	second	lo	hi	Expected result
`['a', 'ab', 'bba']`	`['baa', 'aa', 'bb']`	`5`	`10`	`True`
`['ab', 'bbab', 'bbb']`	`['aba', 'aabb', 'abba']`	`5`	`5`	`False`
`['baa', 'bb', 'bba', 'bbbb', 'a']`	`['b', 'baba', 'baaa', 'aaaa', 'aa']`	`6`	`12`	`True`

Without `lo` and `hi` parameters, this famous problem is algorithmically undecidable: no finite algorithm can guarantee, in finite time, whether a nonempty solution exists for the two given parameter lists in all cases. The problem is **semidecidable**: if a solution exists, systematic breadth-first generation of all solution paths will eventually find one, but there is no general way to determine in finite time whether the current partial solution cannot be extended to a solution.

Bonus problem 113: Optimal bridge placement

```
def optimal_bridges(perm, k):
```

A straight river, uniformly one unit wide, flows from east to west through peaceful land. On the north side of the river are n houses, placed at uniform distances at positions 0, …, n – 1, with an idyllic linear street connecting them along the riverbank. On the south side of the river, *n* workplaces have been similarly positioned at 0, …, *n* – 1 along another linear riverbank street. The homeowner `i` works in the workplace `perm[i]` and is the only homeowner working in that place, so `perm` is guaranteed to be a permutation of integers 0, …, *n* – 1.

Your task is to build exactly *k* bridges across the river. Each bridge must be placed at an exact integer position and cross the river directly to the same position on the other side. Your task is to place these bridges optimally to minimize the sum of distances from the homeowners to their workplace, each homeowner using the bridge that gives him the shortest path to where he is going. Any number of homeowners can share the same bridge on their way to work. The distance travelled by each homeowner `i` to his workplace `perm[i]` is therefore given by the formula `abs(i-b) + 1 + abs(perm[i]-b)`, using the best bridge position `b` for that homeowner. This function should return the minimum total distance across all homeowners.

The automated fuzz tester will give your function enough test cases that are large enough so that brute-forcing through the `combinations(range(n), k)` possible bridge positions will not pass these tests within the set time limit. Instead, loop through all possible groupings of which block of homeowners uses which bridge, then determine where to place that bridge to minimize the total trip for that group.

perm	k	Expected result
`[0, 1]`	2	2
`[4, 2, 1, 0, 3]`	2	15
`[1, 4, 2, 3, 0, 6, 5]`	2	21
`[5, 7, 10, 4, 0, 6, 11, 2, 13, 8, 9, 1, 12, 14, 3]`	3	87

This problem is “Palembang bridges” in the DMOJ problem collection and online judge.

Bonus problem 114: No dogs, no masters

```
def no_dogs_allowed(grid):
```

The earlier problem “Backwards god” had your function count how many times the word `'dog'` appears in the given `grid` of letters. In that problem, the word could appear in any of the eight compass directions and wrap around the edges. However, if we disallow wrapping around the edges and only look at the four cardinal directions, you can always remove all occurrences of 'dog' by swapping letters in pairs, regardless of how they are placed in the original grid. You can swap these letters freely across the entire grid, not just between adjacent letters or letters in the same column or row.

This function should return the smallest number of pairwise swaps necessary to erase all occurrences of the words `'dog'` and `'god'` from the `grid`. As in the previous problem of optimally filling a box of dominoes, symmetries can heavily constrain the search for the optimal solution. For example, consider how swapping all occurrences of `'d'` and `'g'` with each other in the `grid` creates a problem equivalent to the original instance.

grid	Expected result

`['odg',`
` 'ggo',`
` 'dog']`
	1

`['dggodg',`
` 'dooodg',`
` 'ddoggg',`
` 'oododd',`
` 'dddgdo',`
` 'gogogg']`
	2

Bonus problem 115: Car match game

```
def car_match(rows, trucks):
```

The actual steak of a mobile game is seldom as tasty as the sizzle promised in the advertisement video reel. Still, at least the ad for the game “Car Match” gives us a fun combinatorial optimization problem to solve. Each car has a colour, expressed as a nonnegative integer. Several cars in the lot are placed in `rows` so only the car at the end of each row can move out to exit that row, since it blocks the cars behind it.

A series of `trucks`, coloured the same as the cars, will then drive into the lot in the given order. Each truck can carry exactly three cars, all the same colour as that truck. Once a truck has exactly three cars of its colour, it drives away, and the next truck in line can move in to load three cars of its colour. A truck will not drive out of the lot to make room for the next truck until it has exactly three cars. Once you can no longer place three cars of the correct colour on the current truck, the entire lot halts and the game ends.

Your task is to load as many cars as possible onto the `trucks` by tactically choosing each row to `pop` the next car to load onto the truck currently taking cars. This function should return the maximum number of cars that can be removed from the lot under these rules. Again, remember that the last truck will not drive out carrying only one or two cars, so the result will always be divisible by three. The difficulty in this problem is avoiding redundant exploration in the recursion, which can lead to exponential blowup.

rows	trucks	Expected result
`[[0, 0, 2, 0], [1, 1, 1, 0], [1, 0], [1, 0, 0, 0, 0]]`	`[0, 0, 0, 1, 1]`	`6`
`[[3, 3, 4, 0, 1], [2, 3, 2, 4, 3], [0, 0, 1], [2, 3, 4, 4, 1]]`	`[4, 4, 4, 3, 3, 3]`	`0`
`[[2, 1, 3, 3, 3, 3], [2, 3, 3], [2, 2, 2], [2, 1], [1, 3, 3], [0, 1, 1, 1, 1, 1, 3]]`	`[3, 3, 3, 1, 1, 1, 2, 2]`	`15`

Bonus problem 116: Liquid sort

```
def liquid_sort(tubes):
```

Liquid sort is a popular mobile game in which some `tubes` are filled with liquids of different colours that do not mix. Each tube has room for exactly four portions of liquid. Each move pours all consecutive portions of liquid of the same colour from the top of one test tube into another tube that is either empty or also contains liquid of that same colour at the top. The goal is to fill each tube with four portions of liquid of the same colour. Note that in our version of this game, each move must pour all liquid of the same colour into the other tube, and you may not leave any liquid of the colour being poured in the first tube.

Liquid colours are represented as nonnegative integers. Given a list of `tubes`, each tube itself a list whose last element is on the top in the position `-1` (so that you can easily `pop` the portion of liquid from one tube to `append` it to the other), this function should return the maximum number of tubes completed with some optimal sequence of moves. Some of the `tubes` are initially empty to give you some room to maneuver, represented as empty lists.

The main difficulty is efficiently pruning the backtracking search through the search tree of all possible move sequences so the function passes the automated tester within the time limit. Also, don't swish and swosh the same liquid back and forth between a tube filled with only that colour and another tube that is initially empty. All empty tubes can also be treated as the same empty tube at the same level of recursion.

tubes	Expected result
`[[1, 0, 1, 0], [1, 2, 1, 2], [2, 0, 2, 0], [], []]`	3
`[[0, 1, 1, 3], [2, 2, 2, 1], [3, 3, 0, 0], [3, 1, 0, 2], []]`	1
`[[9, 8, 5, 8], [3, 3, 9, 8], [1, 7, 3, 2], [4, 0, 5, 4], [2, 6, 5, 9], [1, 0, 2, 7], [4, 9, 8, 6], [1, 6, 7, 0], [7, 5, 1, 6], [4, 2, 0, 3], []]`	3

Bonus problem 117: Count bowling paths

```
def bowling_paths(score):
```

In the popular game of **ten-pin bowling**, many different sequences of throws can reach the same final score. There is only one way to bowl a dead 0 or a perfect 300, but many different paths lead to the same scores in between. This function should compute the number of different sequences of throws that lead to the same final `score`. For simplicity, we assume no fouls. Gutter balls that knock over zero pins are possible on any throw.

To solve this problem, you need to define a recursive function whose parameters are the current frame number, the score to achieve, and the raw pin results of the next two throws that follow this frame. This function should go through all possible ways to make the two throws in this frame, and recursively count the number of ways to achieve the remaining score in the previous frames. (Note that the frame number **decreases** at each level of recursion, and equals ten at the top-level call.) You also need to handle the last frame separately because of the third bonus throw when there is a strike or a spare. Since the subproblems of this recursion will massively overlap, you should use the `@lru_cache` decorator to memoize the results for fast future access in other recursion branches.

`score`	Expected result
`1`	`20`
`74`	`161209444961060944`
`147`	`46583823560371`
`200`	`1399365362`
`289`	`10`
`290`	`11`
`291`	`1`

Bonus problem 118: Expiring cookies

```
def expiring_cookies(board):
```

An article by Jim Propp examined the following wacky variation of the game of nim. The game state is a `board` of positive integers. Players alternate turns removing any one integer of their choice from the board. Unlike in Nim, the player always takes away that entire number, not just some part of it. After each move, decrement all remaining numbers by one, and remove all zeros. For example, removing one occurrence of 4 from `[1, 2, 4, 4, 6, 7]` leaves `[1, 2, 4, 6, 7]`. Decrementing these values gives `[0, 1, 3, 5, 6]`, and removing all zeros gives the new board `[1, 3, 5, 6]`.

The player who makes the board empty by removing the last remaining number wins the game. This function should determine whether the current player has a winning strategy on the given board. You can determine this with a recursive minimax search that examines all possible moves on the current board and, for each move, recursively computes whether the opponent has a winning strategy on the resulting board. A position is winning for the current player if at least one move leads to a losing position for the opponent.

Since the same board position can be reached in potentially exponentially many ways, maintain sets of known winning and losing boards encountered during the search. When you encounter those boards again, you can return the answer immediately. As you can see from the first two test cases in the table below, unlike ordinary nim, a pair of equal numbers is not a *miai* that cancels out. However, also unlike in ordinary nim, any integer greater than the current length of the board can be considered to be equal to that length. For example, the board `[1, 5, 6, 7]` plays the same as the board `[1, 4, 4, 4]`.

board	Expected result
`[1, 2, 5, 6]`	`True`
`[1, 2, 4, 4, 5, 6]`	`False`
`[1, 1, 2, 4, 6, 8, 10]`	`True`
`[1, 4, 4, 4, 7, 10, 11, 14, 16, 17]`	`False`

Bonus problem 119: Sonata pattern matching

```
def sonata_match(text, pattern):
```

The pattern-matching subroutine of the Turing-complete esoteric programming language Sonata makes for an interesting exercise in Python string processing, along with some associated bookkeeping. This function should determine whether the `pattern` matches the **entire** `text` when all its lowercase characters match themselves, variables of the form `^X` match any single character, and variables of the form `_X` match an arbitrary substring of zero or more characters. In both variable forms, `X` can be any single uppercase letter. If the same variable occurs multiple times inside the `pattern`, that variable must match the same substring of `text` at each occurrence.

This function should return the list of tuples of the form `(variable, match)` of the bindings for all variables of the given `pattern` that give a successful match of the entire `text`. All caret variables must be listed first in alphabetical order, followed by all underscore variables in alphabetical order. If there is no way to match the `pattern` to the entire `text`, this function should return `None`. If the `pattern` can match the entire `text` in different ways, the returned binding should be the one that minimizes the length of the first variable in the `pattern` when read from left to right, followed by the binding of the second variable occurrence that minimizes the length of that variable, and so on.

text	pattern	Expected result
`'tltgt'`	`'^A^B^Ag^A'`	`[('^A', 't'), ('^B', 'l')]`
`'pxplayssplaysf'`	`'px_As_Af'`	`[('_A', 'plays')]`
`'iluikelkkelkkelk'`	`'^Al^Ci_B_B_B'`	`[('^A', 'i'), ('^C', 'u'), ('_B', 'kelk')]`
`'hmaidaqffaaidaeyi'`	`'h_A^Eff_Aeyi'`	`None`

You can write your own recursive matcher function based on a **backtracking** search. Maintain an auxiliary dictionary to keep track of the variable bindings made and unmade during the recursive dive down the `text`. Readers familiar with them can alternatively harness Python's regular expression standard library to do this hard work. (**Backreferences** allow you to ensure that the same part of the regular expression is matched identically in each occurrence of the same variable.)

Bonus problem 120: Fox and hounds

```
def fox_and_hounds(fox, hounds):
```

Fox and hounds is an abstract strategy game whose complexity and state space are small enough that we can calculate the optimal moves for both players in any possible game situation. As explained on the linked Wikipedia page, the game is played on the black squares of an 8-by-8 checkerboard, with the four hounds starting from row zero and aiming to capture one fox starting from row seven. On its turn, a hound can only move forward toward the last row, while the fox can move one step in any of the four diagonal directions. There are no captures, promotions, or jumps, but the fox wins if it reaches row 0, and the hounds win if they block the fox and make it unable to move.

From the starting position, this game is known to be a win for the hounds, but this function should compute whether the `fox` has a winning strategy against the four `hounds` in the given position on his turn to move. The **minimax search algorithm** that can solve not just this but any **complete information two-player game of alternating moves** consists of two **mutually recursive** functions, `fox_move` and `hounds_move`, that each loop through the possible moves for that player in the given situation, and for each move, call the other function to determine if the opponent has a winning strategy in the state resulting from that move. Both functions return `True` if there exists a move for which the opponent has no winning response, and `False` otherwise.

Despite the game's small state space, the space of possible move sequences is still exponentially large, especially since many sequences lead to the same game states. This requires **transposition tables** to store the states encountered during the recursive search. If the search reaches the same game state again, just look up that state's result in the table. The easiest might be to use four sets, `fox_win`, `fox_lose`, `hounds_win` and `hounds_lose`, to store each state after the first evaluation. The reader should still think up additional ways to speed up this algorithm in various places.

fox	hounds	Expected result
`(7, 0)`	`[(0, 1), (0, 3), (0, 5), (0, 7)]`	`False`
`(6, 1)`	`[(1, 2), (3, 2), (4, 3), (4, 5)]`	`False`
`(6, 5)`	`[(0, 5), (2, 1), (3, 0), (5, 6)]`	`True`
`(5, 0)`	`[(1, 2), (2, 7), (3, 0), (3, 4)]`	`False`
`(7, 0)`	`[(1, 0), (3, 6), (4, 7), (5, 4)]`	`True`

Bonus problem 121: Toads and frogs

```
def toads_and_frogs(board):
```

Continuing the theme of complete-information abstract strategy games with alternating moves made by cute animals, toads and frogs produce surprisingly complex situations in combinatorial game theory despite their simple rules and setup. Some toads (`'T'`) and frogs (`'F'`), at least one of each type, are placed on a one-dimensional `board` of squares. On the toad's turn to move, one toad can either move one step to the right to an adjacent empty square (`'.'`), or hop over a frog immediately to its right to an empty square immediately behind that frog. (Unlike in checkers, hopping over a frog does not capture it.) Frogs move the same way, except to the left. If a toad is blocked and cannot move on its turn, it loses the game. The board does not wrap around cyclically, but animals that reach either end of the board are blocked.

This function should determine whether the toads have a winning strategy on the given `board` when it is their turn to move. The two mutually recursive functions for finding the best moves for toads and frogs (or one suitably parameterized function for this symmetric game) are similar to those in the previous fox-and-hounds game. However, transposition tables to memoize game states encountered during the recursive search are even more essential here, since this game is basically nothing but transposed move sequences leading to the same game states. You also need to speed up your code to pass the automated fuzz tester within the time limit.

`board`	Expected result
`'T..F'`	`False`
`'T...T.F..F'`	`False`
`'TTT...FF.F..'`	`True`
`'..T.TT.TTFFFF.F.'`	`True`
`'T.TFTTT.FT..F.FF.F'`	`False`

Finding the winning strategy for toads and frogs is PSPACE-complete, so every algorithm will occasionally run in exponential time, but most instances can be solved in polynomial time.

Bonus problem 122: Unamerican gladiators

```
def gladiators(our, their):
```

Here is another banger from Peter Winkler's "Mathematical Puzzles: A Connoisseur's Collection". Each gladiator has a positive integer strength. When two gladiators with strengths x and y duel, they each win with probabilities of $x / (x + y)$ and $y / (x + y)$ proportional to their respective strengths, so that with lots of luck, even David ($x = 1$) can slay Goliath ($y = 1000$). The winner absorbs his fallen opponent's strength, *Highlander* style, so that his new strength becomes $x + y$.

Two teams of gladiators, `our` valiant heroes versus `their` dastardly villains, take turns choosing one of their gladiators and one of their opponent's gladiators for a duel, continuing until one team is utterly vanquished. If `our` boys go first, and both teams always choose gladiators to maximize their chances of winning, what is the probability that our team of heroes emerges victorious? You should, of course, use exact `Fraction` objects for all calculations and the final result. Only the mission matters, not any individual gladiator's survival chances, so in the best samurai spirit, each gladiator will gladly face even hopeless odds if it helps maximize his team's chances of triumph.

This problem can be solved with **expectimax** search, a variation of minimax search where, at each level of recursion, the consequences of the duels between each possible choice of gladiators for this round, win and lose, are evaluated recursively from the opposing team's point of view, and the possible results are added together weighted by the win/lose probabilities. Then just return the expected value of the best possible choice for the duelling pair at that level. However, as always with expectimax, the search tree branches massively sideways and soon becomes prohibitive. But if the reader adds debugging output for each move's expected value, they should soon notice something interesting that speeds up this search by several orders of magnitude.

our	their	Expected result
`[4, 1]`	`[3, 3]`	`5/11`
`[9, 6, 1]`	`[4, 2, 8]`	`8/15`
`[5, 11, 7, 5]`	`[10, 10, 1, 7]`	`1/2`
`[15, 14, 15, 15, 24]`	`[23, 19, 7, 14, 8]`	`83/154`

Bonus problem 123: Life is like a box of dominoes

`def domino_box(tiles):`

Place a set of n domino `tiles` inside a 2-by-*n* box. None of the domino tiles is a double (that is, both halves of a domino have the same number of pips), but the set can contain copies of identical domino tiles. Placing the halves of two dominoes adjacent to each other so that both halves have an equal number of pips gives you as many points as the number of those pips in each half. The same domino can participate in more than one such pairing with its neighbouring dominoes, but each pairing is counted only once.

Your task is to place these dominoes inside the box to maximize the total score. Each tile can be placed in either orientation, and you can place tiles horizontally or vertically as desired. Since the optimal placement is not necessarily unique, this function should just return this score.

Since there are $Fib(n) \times 2^n \times n!$ different ways to place these dominoes inside the box (DUCY?), far too many to enumerate exhaustively for any nontrivial value of *n*, your search needs to utilize the many explicit and implicit symmetries available in the possible placements of dominoes to allow your code to pass the automated tests within reasonable time. From this Thanksgiving turkey, a product of three exponential quantities, you can keep carving off polynomial slices forever to push the limit of how large instances of this problem you can solve.

`tiles`	Expected result
`[(2, 3), (3, 1), (4, 3), (1, 5)]`	`7`
`[(2, 1), (2, 3), (1, 2), (2, 3)]`	`12`
`[(3, 2), (5, 4), (3, 5), (1, 5), (1, 5), (2, 5), (6, 3)]`	`29`

From the author's first working solution using backtracking search, adding new symmetries always sped up the search by a factor of three to five, so the fuzz tester had to be changed to produce more numerous, longer test cases.